

非凸 α 使用说明

(v251217)

(注意飞行安全需要在有安全防护的情况下进行飞行实验与测试！)

目 录

1. 飞行前准备工作	1
1.1 开箱	1
1.2 硬件准备	2
1.3 遥控器使用说明	2
1.4 在飞行时遇到紧急情况处理	4
1.5 NoMachine 远程连接无人机机载电脑	5
1.6 电池及充电器使用说明	10
1.6.1 电池信息	10
1.6.2 充电器信息	10
1.6.3 充电步骤	11
1.6.4 遥控器电池充电方法	13
2. 无人机安全检查	13
2.1 手动解锁测试	14
2.2 场地安全检查	14
2.3 光流飞行测试	15
2.4 雷达定位测试	15
2.5 双目视觉定位测试（选配功能）	17
2.6 单目视觉检测测试（选配功能）	18
3. 无人机功能模块使用说明	21
3.1 Diff-Planner 自主避障飞行	22
3.1.1 LIO 定位下的 Diff-Planner	22
3.1.2 VIO 定位下的 Diff-Planner（选配功能）	25
3.2 Elastic-Tracker 自主目标跟踪（选配功能）	26
3.3 FUEL 自主探索（选配功能）	29
3.4 Formation 集群（选配功能）	31
附件一 非凸 α 硬件结构图	36
附件二 相关算法资料链接	42
附件三 tag36h11	43

1. 飞行前准备工作

1.1 开箱

收到非凸 α 后，请先核对所发物资飞机及配套材料是否齐全。

1) 检查航空箱外侧、飞机机身和遥控器上端三处标签中的编号是否一致：



2) 检查配套物资是否齐全:



注意: 本图仅为参考, 实际发货物品可能因版本不同而与图片有所差异, 请根据箱子内部的发货清单来核对物资。

1.2 硬件准备

- 1) 飞机本体及其遥控器
- 2) 电池
- 3) 充电器/充电仓
- 4) 用户操作电脑 (Ubuntu 或 Windows 系统均可)

1.3 遥控器使用说明

- 1) 遥控器检查

遥控器在到手时就已经设置好，无需重新设置，为避免误触与防止意外事故发生，仍需检查遥控器拨杆映射及其电量是否正常：

检查遥控器拨杆映射，拨杆对应通道应与下图一致（开机后单击左下角 **End** 按键进入舵量显示界面）：



检查遥控器电量是否正常（再次单击两次 **End** 回到主界面）：



2) 拨杆功能介绍

遥控器各通道拨杆功能图如下所示：





自动起飞状态对应拨杆（进入舵量显示界面，波动拨杆使其舵量显示一致即为自动起飞状态）：



5 通道（第二优先级）：最下（最上为光流定点模式，中间为手控姿态模式，最下为 onboard 模式接入 px4ctrl）

6 通道（第三优先级）：最下（上为悬停模式遥控器控制位置，下为轨迹跟踪模式）

7 通道（第一优先级）：最上（上为解锁电机，下为锁住电机）

8 通道：最下（详见下方说明）

油门：中间位置

8 通道可以用于发送无人机起飞降落命令。另外，在多点规划功能中还可以发送开始多点规划和返程规划命令，在跟踪功能中还可以发送开始跟踪命令，在探索功能中还可以发送开始探索命令。



注意：

- 手持遥控器时，靠近身体一侧定义为通道上侧，远离身体一侧定义为通道下侧。
- 光流模式必须在光线良好且地面纹理充足的情况下使用
- 6 通道轨迹跟随模式和悬停模式需要在 5 通道开启 onboard 模式且启动定位程序时才能使用，当上位机没发送轨迹指令时无人机保持悬停。

1.4 在飞行时遇到紧急情况处理

1) 定位未崩溃，轨迹跟踪异常

现象：飞机姿态基本稳定，不期望继续跟踪轨迹

处理措施：**6通道上拨，自稳停于原地**，通过下拨油门杆，缓慢降低飞机高度。在合适较低高度处，油门保持最低位，拨下7通道，紧急停桨，然后终止所有程序。

2) 定位崩溃，但遥控器操作手飞行技术良好

现象：飞机姿态剧烈变化，严重失控

处理措施：**上拨5通道，手控救回飞机**。

3) 定位崩溃，但遥控器操作手飞行技术一般

现象：飞机姿态剧烈变化，严重失控

处理措施：**下拨7通道，紧急停桨**。

1.5 NoMachine 远程连接无人机机载电脑

1) 无人机上电

电池放置注意事项：

a) 将电池从起落架开口放入，放入时需保证水平放置飞机，同时上端两侧精准卡在凹槽中。

b) 按住电池上端卡扣将电池平稳推到底。

c) 感受到电池插入并听到电调的开机提示音，即可完成开机。



注意：当电池没电时飞机会长鸣报警，请及时下电，以免电池过放报废；若飞行途中发出报警请及时停止飞行任务，执行降落。飞机上电一段时间，电调会自检并发出有规律的滴滴声，属于正常现象，注意与长鸣报警相区分。

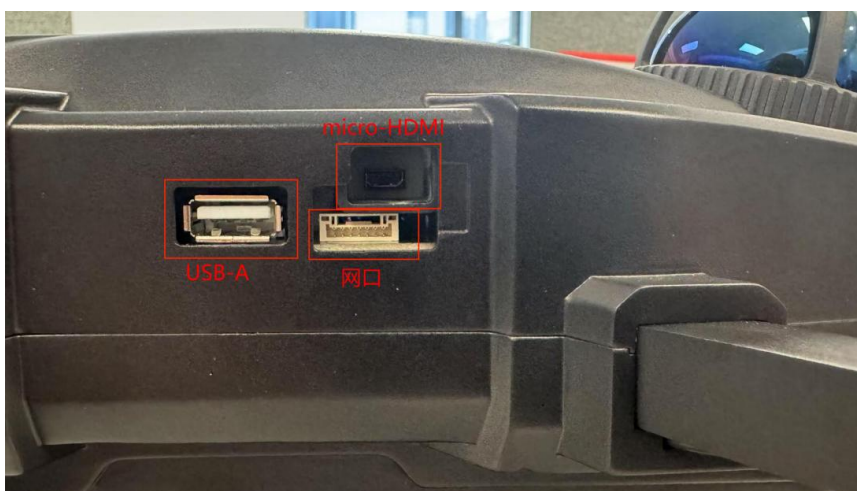
2) 先连接 mini HDMI 线至显示器，给无人机供电，插入 type-A 拓展坞连接键盘和鼠标。

注意：我们建议使用 type-A 拓展坞连接键盘鼠标，使用 type-C 拓展坞可能会偶尔导致设备供电不足。

飞机接口图片及其标识：



(飞机左侧接口图)



(飞机右侧接口)

3) 进入无人机 ubuntu 系统界面，将机载计算机连上路由器的 WiFi，并且在终端输入 ifconfig 指令查看飞机设备 IP。

```

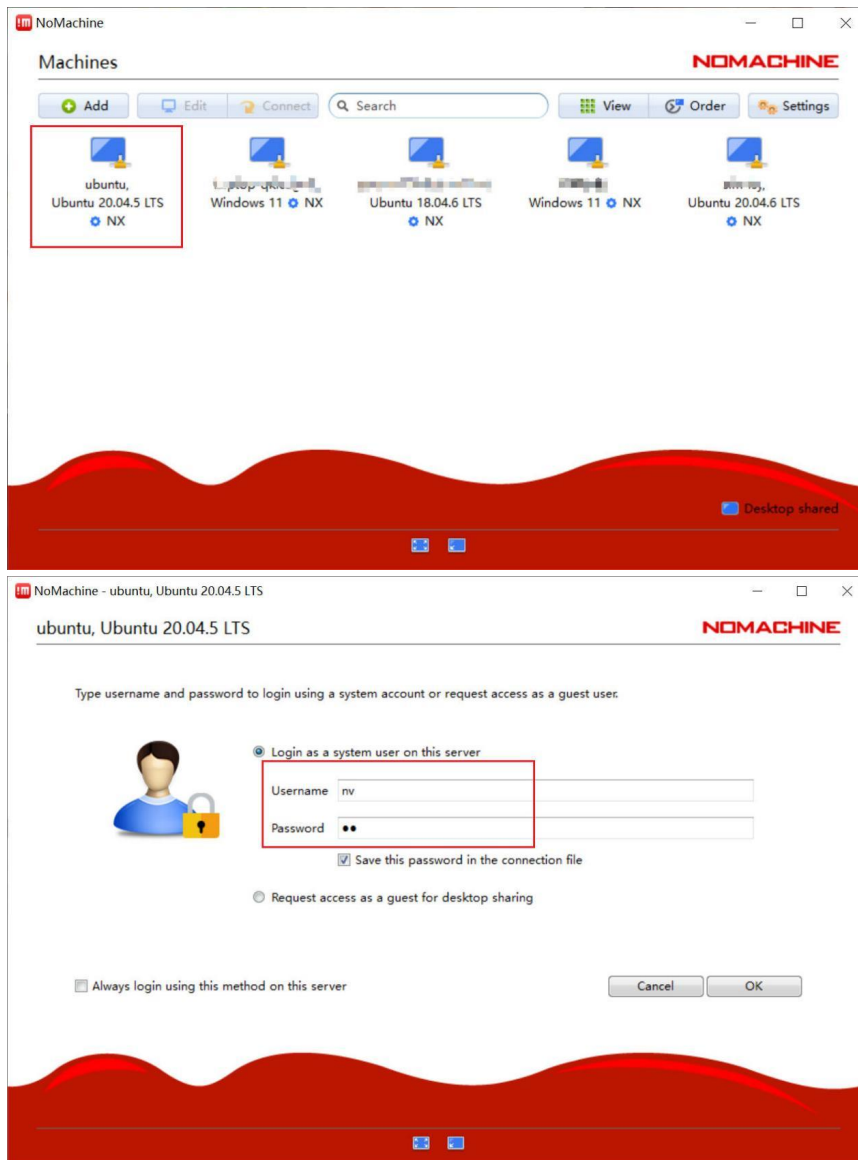
nv@ubuntu:~ 65x16
ether a2:95:f9:dd:49:6b txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions
wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 192.168.3.174 netmask 255.255.255.0 broadcast 192.168.3.255
inet6 fe80::2bea:249e:f8e6:bda6 prefixlen 64 scopeid 0x20<link>
inet6 240e:3b1:d00d:8df3:4d65:8fce:cb41:5049 prefixlen 64 scopeid 0x0<global>
inet6 240e:3b1:d00d:8df3:ad77:cfc9:c86:48a0 prefixlen 64 scopeid 0x0<global>
inet6 240e:3b1:d00d:8df3:7804:e312:d7e1:14 prefixlen 128 scopeid 0x0<global>
nv@ubuntu:~ 65x17
  
```

4) 将笔记本连上相同路由器的 WiFi，打开 NoMachine，此时会自动搜索到可连接的设备，双击点击，通过 NoMachine 连接到飞机计算机的远程桌面。（用户名与密码

即为 nv) 此处连接后若黑屏：使用插入附赠的 HDMI-micro 转 HDMI 转接线，再插入显卡欺骗器即可解决。

若第一次使用，先安装 NoMachine:

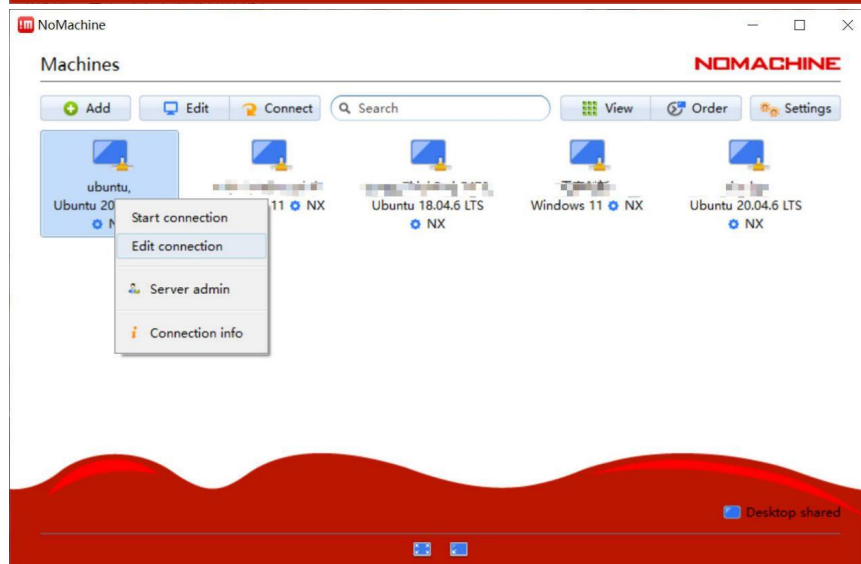
- windows 下载链接: <https://downloads.nomachine.com/windows/?id=3>
- Linux 下载链接: <https://downloads.nomachine.com/linux/?id=1>

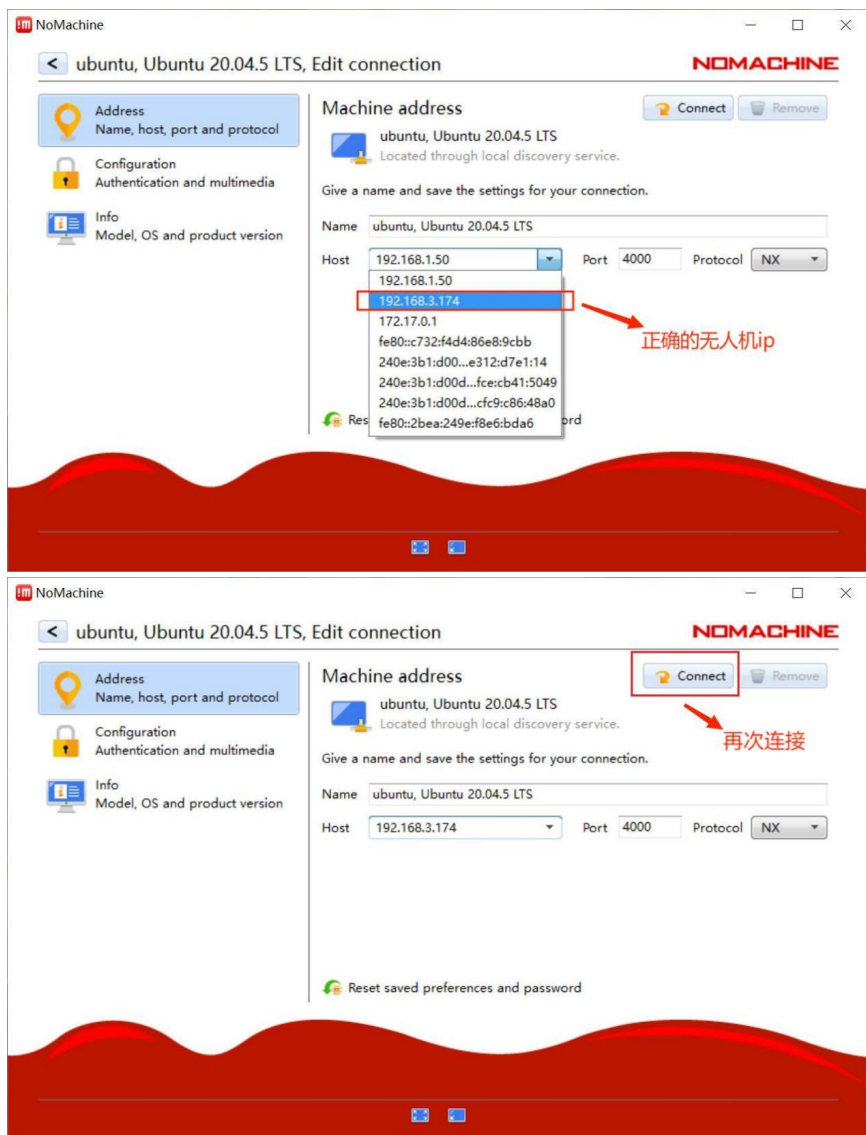


5) 假若出现下图双击后无法进行连接的情况，是因为搜寻到的 IP 与无人机的不符。先右键点击设备名称，点击 **Edit connection**，选择正确的无人机 IP（一般是选择框的第二个 IP），再次进行连接即可。



Connecting to ubuntu, Ubuntu 20.04.5 LTS





6) 在连接过一次之后，只要无人机机载设备没有切换 wifi，下次使用可以直接打开 nomachine 进行远程连接，不需要再去连接显示器查找 ip。

注意：非凸 α 在出厂时已对系统进行特调，无需显卡欺骗器即可直连 NoMachine。若连接黑屏，可以使用赠送的显卡欺骗器。

1.6 电池及充电器使用说明

1.6.1 电池信息

电池信息	
	
电池类型	锂聚合物电池 (Li-Po)
电池容量	6000 mAh
电池能量	136.8 Wh
额定电压	25.2 V

1.6.2 充电器信息

HOTA D6 Pro:



功能参数	
输入电压	AC100~240V/DC6.5~30V
充电电流	0.1~15A x2
充电功率	DC 325W x2 AC 200W(自动分配)
放电功率	内部放电:15Wx2(平衡口 10Wx2) 外部放电:325Wx2(650W)
电池类型	LiHv/LiPo/LiFe/Lilon/Lixx :1~6S NiZn/Nicd/NiMH :1~16S Smart Battery :1~6S Lead Acid(Pb) :2~24V

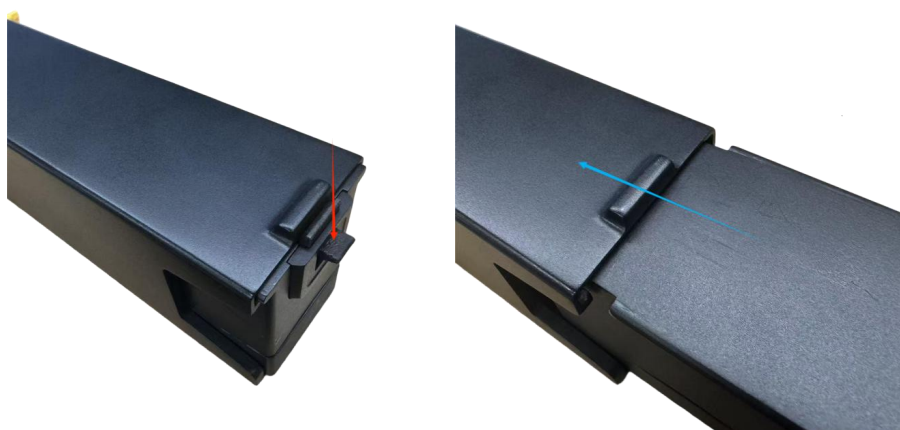
	Enelop :1~16S
平衡电流	最大 1600mA $\times$ 2
放电电流	0.1~3A $\times$ 2
外部放电电流	1~15A $\times$ 2
USB 输出口	5V/2.1A
存储温度	-20~60°C
工作温度	0~40°C
显示屏尺寸	2.8"320x240 26 万色
外形尺寸	108mmx105mmx76mm

1.6.3 充电步骤

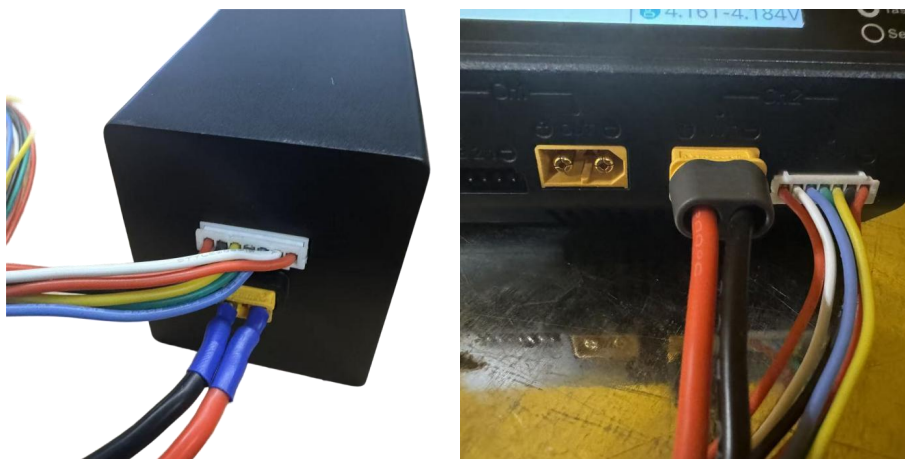
1) 在防爆电池柜中使用 AC IN 口给充电器供电，使充电器开机：



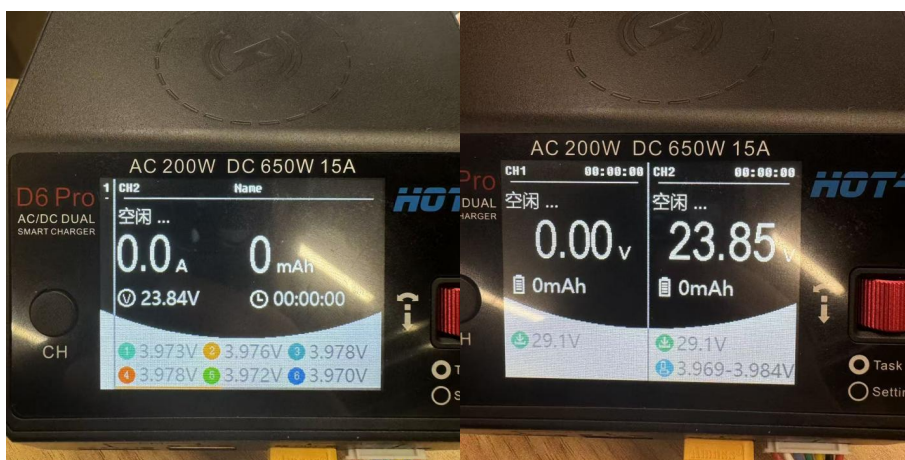
2) 按住电池上方卡扣，将电池平行推入电池充电仓，将电池牢牢卡住：



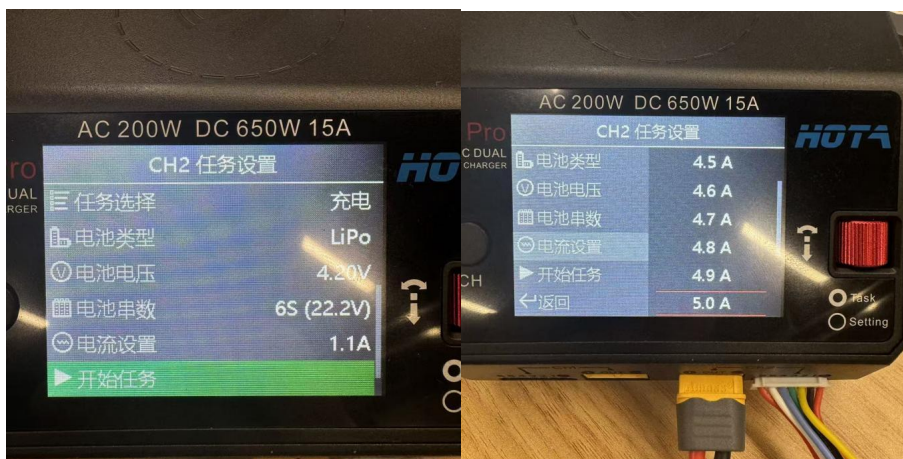
3) 将充电线 and 平衡头分别插入充电仓和充电器上：



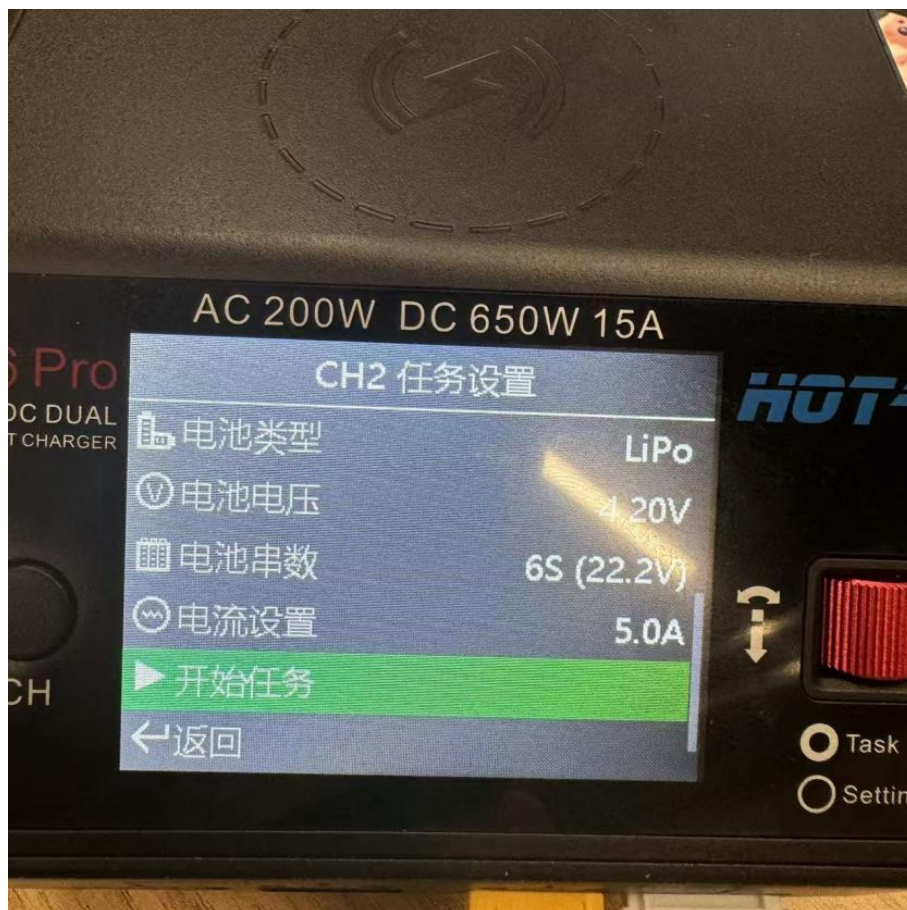
4) 充电器显示电压时即为准备完毕，此时点击左侧 CH 按钮选择通道，选择后可查看每节电池电压：



5) 点击右侧红色按钮，进入 CH2 任务设置后左右滑动红色按钮选择电流设置，将其修改为 5.0A：



6) 点击开始任务充电，当电流有明显降低或归零时即为满电，此时结束任务并拔下充电器即可。



1.6.4 遥控器电池充电方法

遥控器电池充电使用配套的充电转换头，USB 线连接自己的适配器或者电脑 USB 口，当所有充电指示绿灯都亮起即充电完毕。



2. 无人机安全检查

注意：在进行任何飞行操作前，请确保无人机已正确安装桨叶保护罩，飞行场地已设置安全防护网，并确保飞手具备充分的操控能力与应急处置能力。所有飞行活动必须在安全防护措施完善的前提下进行。

2.1 手动解锁测试

目标：

- 1) 测试电机、电调是否正常。
- 2) 测试飞控及遥控器是否正常。

步骤：

- 1) 将无人机上电并放置于空旷平整地面，机头朝前摆放。
- 2) 确保遥控器正常连接且与飞机对应。



3) 测试遥控器解锁和上锁功能。将7通道拨到上方，把左侧摇杆打到右下方并保持以解锁飞机，此时无人机桨叶开始怠速转动；之后将左侧摇杆打到左下方并保持，桨叶上锁即完成测试。**注意：解锁和上锁过程中不要抬起油门。**

4) 测试遥控器急停功能。将7通道拨到上方，把左侧摇杆打到右下方并保持以解锁飞机，之后将7通道拨下锁桨即可完成测试。**注意：此过程中不要抬起油门，且7通拨下以后在5s内不能上拨，否则无人机会再次解锁！**

2.2 场地安全检查

目标：

确保在使用雷达传感器进行定位与飞行前，场地具备完善的安全防护措施，防止

人员受伤或设备损坏。

安全警告：

无人机属于高能、高速旋转设备，具备强动力系统与高速桨叶，一旦操作不当或场地防护不足，可能造成严重人身伤害或财产损失。任何飞行测试前，务必严格执行以下安全检查：

- 1) 确认防护网、隔离栏完好且覆盖全部飞行范围；
- 2) 确认场地的消防设备齐全，有灭火器、应急电源和急救箱并保持可用；
- 3) 确认飞行过程中没有无关人员会突然出现在飞行场地内。

2.3 光流飞行测试

目标：

- 1) 测试光流传感器是否正常工作。
- 2) 测试光流悬停是否稳定。

步骤：

1) 选择光线充足且地面纹理良好（非光滑反光平面）的实验场地进行飞行，测试时请保证飞机已携带桨叶保护罩，同时飞行场地四周已安装安全防护网。

2) 确保遥控器正常连接且与飞机对应。此时 5 通道在最上（光流模式），7 通道在上（解锁），确保飞机下方光流传感器未被遮挡。



3) 手动解锁飞机，缓慢向上推油门至中位，并继续缓缓向上推油门直至飞机离地，光流定位模式下需至少推至 60% 油门才可起飞。起飞后简单移动习惯飞机的操控即可降落。

2.4 雷达定位测试

目标：

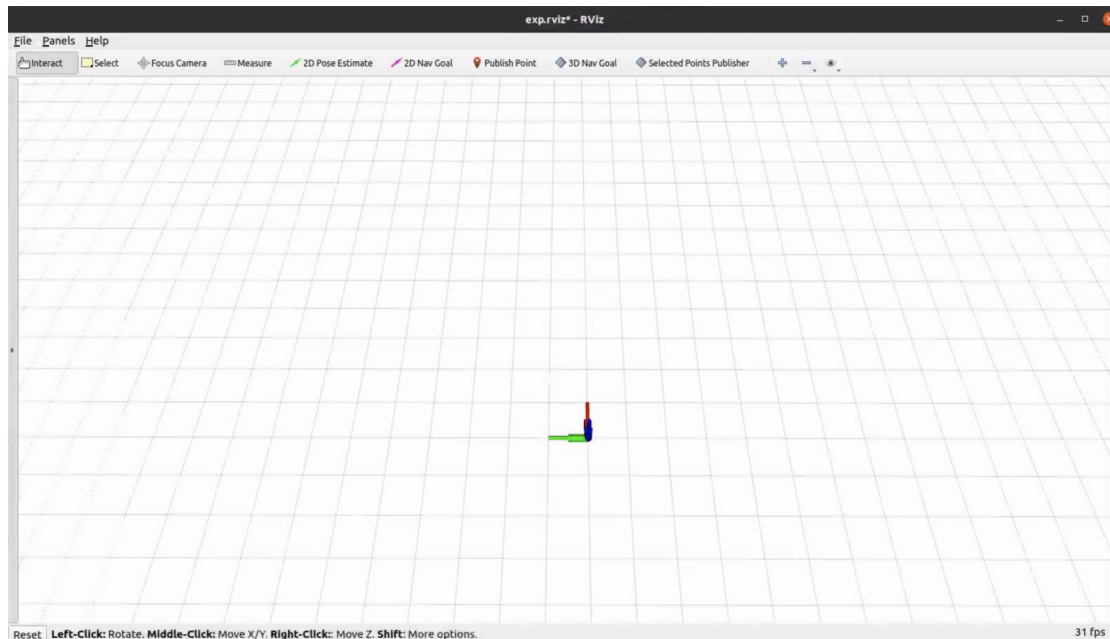
- 1) 测试激光雷达是否正常工作。
- 2) 测试雷达定位是否稳定，为基于雷达定位的自主飞行做准备。

步骤：

1) 将无人机放置于地面同时标记好无人机位置，进入 Diff-Planner 工作空间，启动 FAST-LIO 定位

```
./sh files/run_luo.sh
```

2) 启动后等待 20s, 待节点全部启动。启动过程中注意查看控制台中是否会有报错, 启动完成后可以在 rviz 界面看到无人机当前位置:

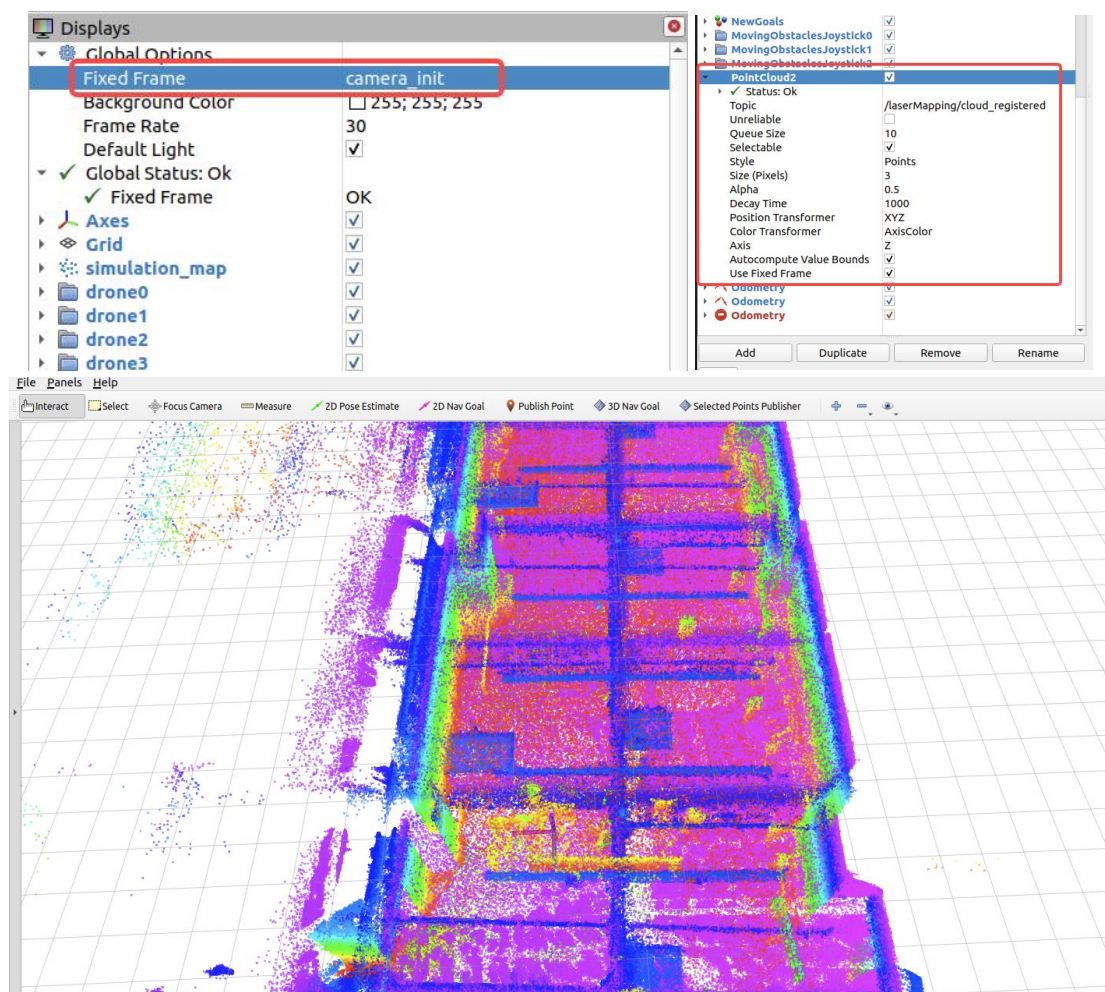


3) 输入指令：`rostopic echo /ekf/ekf_odom` 查看飞机当前的 odom 信息，输入后可在控制台中看到飞机当前的坐标位置：

```
seq: 1533  
stamp:  
  secs: 1760598104  
  nsecs: 249644974  
frame_id: "world"  
child_frame_id: ''  
pose:  
  pose:  
    position:  
      x: -0.0030298474455208026  
      y: -0.0009056173328057553  
      z: 0.00039983451480679854  
    orientation:  
      x: 0.00041141764094108007  
      y: 0.00044042855554779180  
      z: -7.619309718669609e-05  
      w: 0.9999998154763950  
  covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,  
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]  
twist:  
  twist:  
    linear:  
      x: 0.004278391387234906  
      y: 0.001248931229372236  
      z: 0.005114747807869804  
    angular:  
      x: 0.0  
      y: 0.0  
      z: 0.0  
  covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,  
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

4) 拿起飞机，绕飞行场地缓慢移动。移动时可以看到 **rviz** 画面和控制台中的无人机坐标同时移动。绕场一周后将无人机放回原点。再次查看控制台中无人机的定位数据，如果当前坐标偏差在合理的范围内 (0.1m) 即可认为定位稳定。

5) 可在 rviz 界面通过如下设置查看雷达点云质量:



2.5 双目视觉定位测试（选配功能）

目标：

- 1) 测试双目相机是否正常工作。
- 2) 测试视觉定位是否稳定，为基于视觉定位的自主飞行做准备。

注意：

视觉定位对环境特征要求较高，尽量在飞行场地多放一些障碍物增加视觉特征点，避免单一颜色的背景。

步骤：

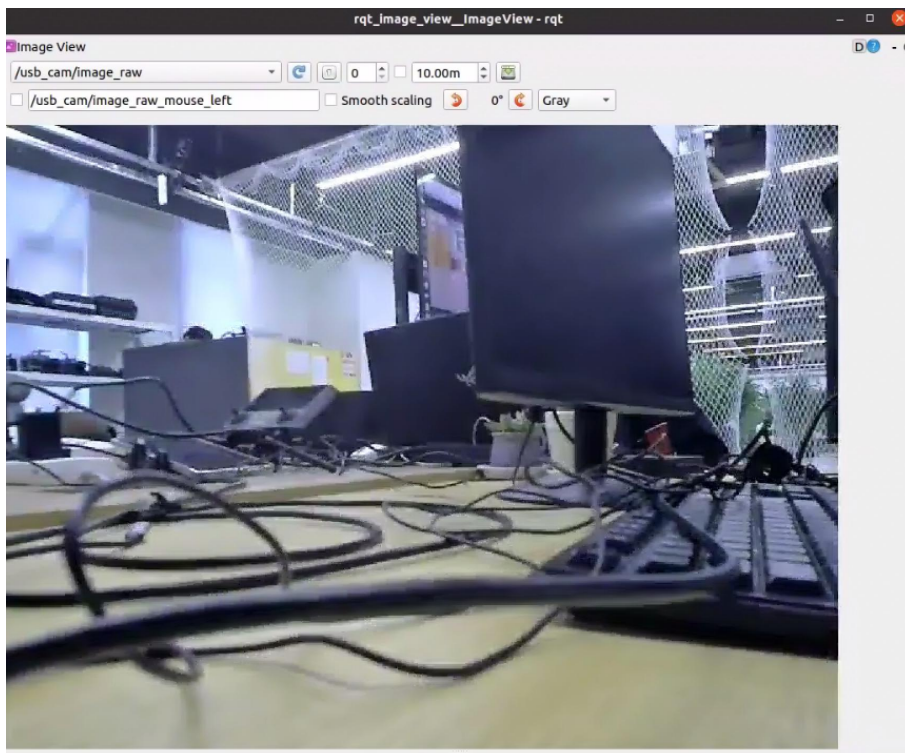
1) 将无人机放置于地面同时标记好无人机位置，进入 Diff-Planner 工作空间启动 VINS 定位

```
cd Diff-Planner
./sh_files/run_vins.sh
```

启动后等待 20s，待节点全部启动。启动过程中注意查看控制台中是否会有报错，启动完成后同样可以在 rviz 界面看到无人机当前位置。

2) 输入指令：`rostopic echo /vins/imu_propagate` 查看飞机当前的 odom 信息，输入后可在终端中看到飞机当前的坐标位置：

2) 使用 `rqt_image_view` 指令打开图像可视化工具，选择 `/usb_cam/image_raw` 图像话题即可看到单目相机图像画面



若启动相机驱动过程中出现 **Error starting device** 错误，或者 `/usb_cam/image_raw` 话题对应图像源不是单目的，需要检查 `src/other-lib/usb_cam/launch/usb_cam.launch` 文件中 `video_device` 对应的端口号是否对应 USB 相机占用端口。

```
usb_cam.launch x
src > other-lib > usb_cam > launch > usb_cam.launch
1 <launch>
2 <node name="usb_cam" pkg="usb_cam" type="usb_cam node" output="screen" >
3   <param name="video_device" value="/dev/video0"/>
4   <param name="image_width" value="640" />
5   <param name="image_height" value="480" />
6   <param name="pixel_format" value="yuyv" />
7   <param name="color_format" value="yuv422p" />
8   <param name="camera_frame_id" value="usb_cam" />
9   <param name="io_method" value="mmap"/>
10  <param name="framerate" value="30"/>
11 </node>
12 <!-- <node name="image_view" pkg="image_view" type="image_view" respawn="false" output="screen">
13   <remap from="image" to="/usb_cam/image_raw"/>
14   <param name="autosize" value="true" />
15 </node> -->
16 </launch>
```

3) 新建一个终端打开二维码检测脚本

```
cd ~/Elastic-Tracker
source devel/setup.zsh
roslaunch apriltag_ros continuous_detection.launch
```

```

/home/nv/Elastic-Tracker/src/detection/apriltag_ros/apriltag_ros/launch/continuous_detection.launch ht
* /apriltag_ros_continuous_node/cy: 257.1316496683269
* /apriltag_ros_continuous_node/fx: 339.2910715400404
* /apriltag_ros_continuous_node/fy: 326.504458217146
* /apriltag_ros_continuous_node/max_hamming_dist: 2
* /apriltag_ros_continuous_node/publish_tag_detections_image: True
* /apriltag_ros_continuous_node/publish_tf: True
* /apriltag_ros_continuous_node/standalone_tags: [{'id': 0, 'size'...
* /apriltag_ros_continuous_node/tag_blur: 0.0
* /apriltag_ros_continuous_node/tag_bundles: [{'name': 'CUSTOM...
* /apriltag_ros_continuous_node/tag_debug: 0
* /apriltag_ros_continuous_node/tag_decimate: 1.0
* /apriltag_ros_continuous_node/tag_family: tag36h11
* /apriltag_ros_continuous_node/tag_refine_edges: 1
* /apriltag_ros_continuous_node/tag_threads: 2
* /apriltag_ros_continuous_node/transport_hint: raw
* /rostdistro: noetic
* /rosversion: 1.17.0

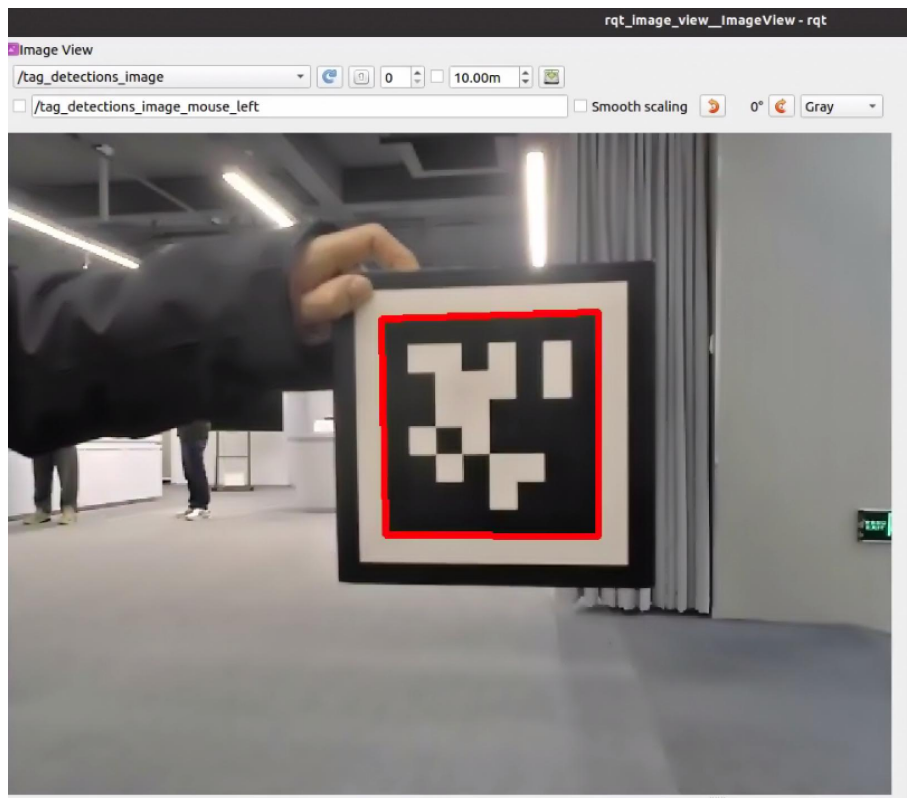
NODES
/
  apriltag_ros_continuous_node (apriltag_ros/apriltag_ros_continuous_node)

ROS_MASTER_URI=http://localhost:11311

process[apriltag_ros_continuous_node-1]: started with pid [15992]
[ INFO] [4587.489265527]: Initializing nodelet with 8 worker threads.
[ INFO] [4588.149193009]: Loaded tag config: 0, size: 0.13, frame_name: tag0
[ INFO] [4588.149834855]: Loading tag bundle 'CUSTOM_BUNDLE_NAME'
[ INFO] [4588.149911369]: 0) id: 0, size: 0.13, p = [0,0,0], q = [0,0,0,0]
[ WARN] [4588.150837800]: remove_duplicates parameter not provided. Defaulting to
true
[ INFO] [4588.411500363]: Camera model: fx = 540.179, fy = 541.365, cx = 284.989,
cy = 228.9

```

4) 在 `rqt_image_view` 中查看 `/tag_detections_image` 话题图像，并将事先打印好的二维码放到相机前，查看是否能稳定识别到二维码（标记为红框）。



二维码的种类和尺寸分别在/src/detection/apriltag_ros/apriltag_ros/config 文件夹下的 settings.yaml 和 tags.yaml 中设置：

```
! settings.yaml x
src > detection > apriltag_ros > apriltag_ros > config > ! settings.yaml
1 # AprilTag 3 code parameters
2 # Find descriptions in apriltag/include/apriltag.h:struct apriltag_detector
3 # apriltag/include/apriltag.h:struct apriltag_family
4 tag_family: 'tag36h11' # options: tagStandard52h13, tagStandard41h12, tag36h11, tag
5 tag_threads: 2 # default: 2
6 tag_decimate: 1.0 # default: 1.0
7 tag_blur: 0.0 # default: 0.0
8 tag_refine_edges: 1 # default: 1
9 tag_debug: 0 # default: 0
10 max_hamming_dist: 2 # default: 2 (Tunable parameter with 2 being a good choice -
11 # Other parameters
12 publish_tf: true # default: false
13 transport_hint: "raw" # default: raw, see http://wiki.ros.org/image_transport#Know
14 # fx: 497.6472523273357
15 # cx: 312.16452949230853
16 # fy: 497.4291262877136
17 # cy: 209.88176426745628
18 fx: 339.2910715400404
19 cx: 340.9384810422289
20 fy: 326.504458217146
21 cy: 257.1316496683269
```

```
! settings.yaml ! tags.yaml x
src > detection > apriltag_ros > apriltag_ros > config > ! tags.yaml
17 # standalone_tags:
18 # [
19 #   {id: ID, size: SIZE, name: NAME},
20 #   ...
21 # ]
22 standalone_tags:
23 [
24   {id: 0, size: 0.13, name: tag0},
25 ]
26 ## Tag bundle definitions
27 ### Remarks
28 #
29 # - name is optional
30 # - x, y, z have default values of 0 thus they are optional
31 # - qw has default value of 1 and qx, qy, qz have default values of 0 thus they are optional
32 #
33 ### Syntax
34 #
35 # tag_bundles:
36 # [
37 #   {
38 #     name: 'CUSTOM_BUNDLE_NAME',
39 #     layout:
40 #     [
41 #       {id: ID, size: SIZE, x: X_POS, y: Y_POS, z: Z_POS, qw: QUAT_W_VAL, qx: QUAT_X_VAL, qy: QUAT_Y_VAL, qz: QUAT_Z_VAL},
42 #       ...
43 #     ]
44 #   },
45 #   ...
46 # ]
47 tag_bundles:
48 [
49   {
50     name: 'CUSTOM_BUNDLE_NAME',
51     layout:
52     [
53       {id: 0, size: 0.13, x: 0.0000, y: 0.0000, z: 0.0000, qw: 0.0000, qx: 0.0000, qy: 0.0000, qz: 0.0000},
54     ]
55   },
56 ]
```

二维码图片下载和尺寸测量方法参考：<https://github.com/AprilRobotics/apriltag?tab=readme-ov-file#choosing-a-tag-family>。附件三提供了本例中使用的二维码图像，打印时注意长宽比和图片中保持一致，并测量实物尺寸后填入 tags.yaml。

3. 无人机功能模块使用说明

注意：在进行任何飞行操作前，请确保无人机已正确安装桨叶保护罩，飞行场地已设置安全防护网，并确保飞手具备充分的操控能力与应急处置能力。所有飞行活动必须在安全防护措施完善的前提下进行。

3.1 Diff-Planner 自主避障飞行

目标：

- 1) 完成 Diff-Planner 自主飞行功能测试，实现自主飞行避障。
- 2) 熟悉自主飞行参数。

3.1.1 LIO 定位下的 Diff-Planner

进行前请先完成 2.4 节雷达定位测试，确保雷达定位无问题再进行！

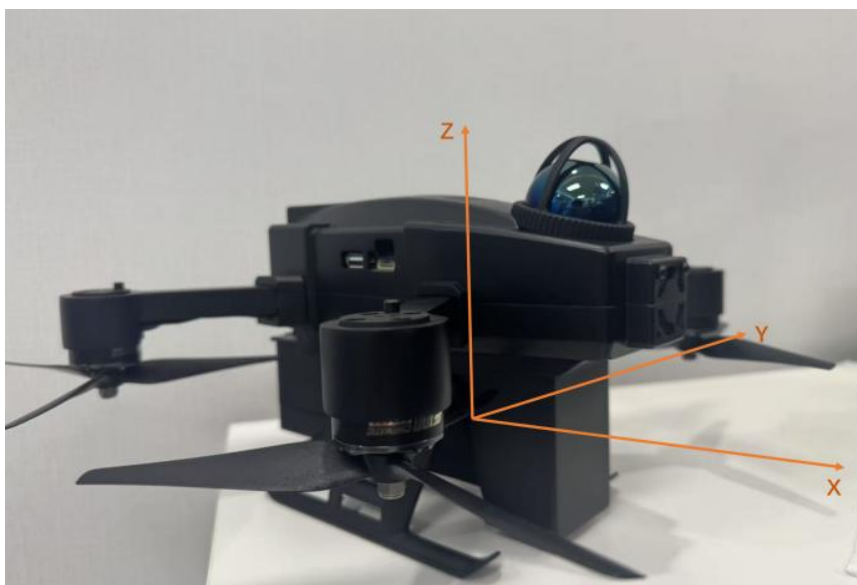
步骤：

- 1) 在配置文件 `src/user_command/multipoint/launch/multipointplan_exp_lio.launch` 中查看设置的飞行模式

```
src > user_command > multipoint > launch > multipointplan_exp_lio.launch
1  <launch>
2    <arg name="odom_topic" value="/ekf/ekf_odom" />
3    <arg name="next_distance" value="0.7" />
4    <arg name="flight_type" value="1" /> <!-- 1到4代表4种控制模式 -->
5    <arg name="start_plan" value="1" /> <!-- 1表示启动多点规划，0表示不启动 -->
6    <arg name="back_plan" value="1" /> <!-- 1表示启动回城多点规划，0表示不启动 -->
7
8    <node pkg="multipoint" name="multipointplan" type="multipointplan" output="screen">
9      <remap from="odom_topic" to="$(arg odom_topic)" />
10     <param name="yaml_path" value="$(find multipoint)/config/points.yaml" />
11     <param name="next_distance" value="$(arg next_distance)" />
12     <param name="flight_type" value="$(arg flight_type)" />
13     <param name="start_plan" value="$(arg start_plan)" />
14     <param name="back_plan" value="$(arg back_plan)" />
15   </node>
16 </launch>
```

`flight_type` 为 1 表示选择 `test1` 模式进行多点规划，即依次经过各途经点；`flight_type` 为 2 表示选择 `test2` 模式进行多点规划，可以控制到达各途径点后的停留时间。其余模式的定义可在 `src/user_command/multipoint/config/points.yaml` 中找到。

- 2) 在 `src/user_command/multipoint/config/points.yaml` 中设置 `test1` 对应的途径点坐标，以及返程规划的途径点坐标。点位坐标系以飞机起飞位置为 $(0, 0, 0)$ 原点，无人机体系 x 轴方向为点位 x 轴正方向，无人机坐标系：



点位需要按照特定格式修改。每个点通过 `- [x y z]` 确认，分别表示点位 $x/y/z$ 坐标

(单位为米)。

```
src > user_command > multipoint > config > ! points.yaml
1 #使用说明 1.该功能包提供4种多点多控制的模式: 1.只进行多点点规划 3.进行多点点规划并且实现每个点的停留时间控制 3.进行多点点规划并且实现yaw角控制 4.进行多点点规划并且实现每个点的停留时间控制以及yaw角控制
2 #
3 #
4 #
5 #模式1 x y z
6 test1:
7   - [8.0, 0.0, 1.2] # Point 1
8   - [1.0, 0.5, 1.2] # Point 2
9   - [8.0, -0.2, 1.2] # Point 3
10
11 #模式2 x y z time
12 test2:
13   - [8, 0.0, 1.2, 2] # Point 1
14   - [0, 8.0, 1.2, 0] # Point 2
15   - [0, -2.0, 1.2, 0] # Point 3
16
17 #模式3 x y z yaw
18 test3:
19   - [8, 0.0, 1.0, 3] # Point 1
20   - [0, 8.0, 1.2, 4] # Point 2
21   - [0, -2.0, 1.2, 3] # Point 3
22
23 #模式4 x y z yaw time
24 test4:
25   - [8.0, 0.0, 1.0, 1.57, 0] # Point 1
26   - [0.5, 0.0, 1.0, -100, 0] # Point 2
27
28 test back:
29   - [0, 0, 1.0]
```

3) 将无人机放置于安全、平整地面,以无人机为中心半径 50cm 的四周请勿放置任何障碍物。打开遥控器,保持 5 通、6 通、8 通在最下通道,7 通拨杆在上,油门保持中位。



4) 检查 src/realflight_modules/px4ctrl/config/ctrl_param_fpv.yaml 中起飞高度 takeoff_height 设置是否合适

```
! ctrl_param_fpv.yaml x
src > realflight_modules > px4ctrl > config > ! ctrl_param_fpv.yaml
1 #####
2 # ★★★ Set parameters carefully for satisfied performance! ★★★ #
3 #####
4
5 mass      : 1.0 # kg
6 gra       : 9.81
7 pose_solver : 0 # 1:From ZhepeiWang, 2:From rotor-drag
8 ctrl_freq_max : 200.0
9 max_manual_vel: 1.0
10 max_angle: 80 # Attitude angle limit in degree. A negative value means no limit.
11 low_voltage: 21.0 # 6S battery
12
13 rc_reverse: # *
14   roll: false
15   pitch: true
16   yaw: false
17   throttle: true
18
19 auto_takeoff_land:
20   enable: true
21   enable_auto_arm: true
22   no RC: false
23   takeoff height: 0.8 # m
24   takeoff_land_speed: 0.2 # m/s
```

5) 进入 Diff-Planner 工作空间并启动雷达定位导航脚本

```
cd ~/Diff-Planner
./sh_files/run_single_lio.sh
```

待程序完全启动后 rviz 界面如图所示：



注意：若没有执行规划任务需求只是测试程序，请保持 7 通拨杆在下进入急停状态或关闭遥控器，以免误触遥控器 8 通导致无人机自动起飞！

6) 起飞：确定飞机所在环境安全无人员靠近并再次确定遥控器拨杆状态正常。把遥控器的 8 通道从下挡拨至中档（或在终端进入工作空间并输入./sh_files/takeoff.sh），此时无人机会自主起飞，悬停于设置的高度处。此状态下仍可用遥控器操控飞机（雷达定位模式）。

注意：若未提前 5 通道、6 通道下拨，请关闭程序，重新调整遥控器档位。切勿强行起飞，否则可能会油门估计异常！遥控器 8 通和终端都可以发送起飞、执行任务、返航、降落指令，每次任务只能选择同一种发指令方式，以免重复发送命令！

7) 执行规划任务：8 通道打到上位（或在终端进入工作空间并输入./sh_files/pub_trigger.sh），无人机开始自动规划至目标点，依次到达目标点后进入悬停。

8) 返程规划：到达最后一个的目标点后，待无人机稳定悬停，把遥控器的 8 通道从上档拨至中档（或在终端进入工作空间并输入./sh_files/back.sh），可以按照预先写好的点位进行返程规划，在到达最后一个返程途径点后进入悬停。

9) 结束任务：功能使用完毕，把 8 通道从中档拨至下档（或在终端进入工作空间并输入./sh_files/land.sh），无人机开始自动降落；也可以上拨 6 通道进入自主悬停模式，轻轻下拨油门通道，无人机会缓慢下降。待无人机落地后，上拨 7 通道锁桨，并在任务终端输入 ctrl+c 结束任务。

10) 参数说明：规划和建图模块的基础参数位于 src/diff_planner/plan_manage/launch/exp/run_exp_single_lio.launch 中，各参数含义如下表所示：

run_exp_single_lio.launch 参数表		
参数名	参数值	说明
odom_topic	/ekf/ekf_odom	里程计话题名称

drone_id	\$(env DRONE_ID)	无人机编号，在 run_single_lio.sh 中设置
camera_pose_topic	no_use0	相机位姿话题（未使用）
depth_topic	/camera/depth/image_rect_raw	深度图话题（未使用）
cloud_topic	no_use2	激光点云话题（未使用）
cx	-	相机内参 cx，雷达定位中未使用
cy	-	相机内参 cy，雷达定位中未使用
fx	-	相机内参 fx，雷达定位中未使用
fy	-	相机内参 fy，雷达定位中未使用
max_vel	0.5	最大速度(m/s)
max_acc	3.0	最大加速度(m/s ²)
max_jer	20.0	最大加加速度(m/s ³)
planning_horizon	7.5	规划范围（建议为感知范围的 1.5 倍）
local_map_size_x	5.5	局部地图 x 轴方向尺寸(m)
local_map_size_y	5.5	局部地图 y 轴方向尺寸(m)
local_map_size_z	2.0	局部地图 z 轴方向尺寸(m)
enable_virtual_wall	true	是否启用虚拟天花板和地面
virtual_ceil	2.0	虚拟天花板高度(m)
virtual_ground	0.1	虚拟地面高度(m)
map_resolution	0.15	地图分辨率(m)
inflation_size	0.15	障碍物膨胀尺寸(m)
flight_type	1	目标点模式，1 表示接收 rviz 目标点或者 user_command 目标点，2 表示使用预设的全局目标点
point_num	2	全局目标点数量，只在 flight_type=2 时用到
pointk_x/y/z	-	第 k 个全局目标点的 x/y/z 坐标，只在 flight_type=2 时用到

3.1.2 VIO 定位下的 Diff-Planner（选配功能）

VIO 定位下的 Diff-Planner 点位设置、遥控器拨杆和任务执行方式和使用雷达定位的相同，进行前请先阅读 3.1.1 节并完成 2.5 节双目视觉定位测试。飞行模式的配置文件位于 src/user_command/multipoint/launch/multipointplan_exp_vio.launch，规划和建图模块的基础参数位于 diff_planner/plan_manage/launch/exp/run_exp_single_vio.launch。注意：基于视觉定位的规划只能选择 test1 和 test2 模式，即默认无人机 yaw 角对准前进方向，请不要尝试可以自定义 yaw 角的 test3 和 test4 模式，很容易导致无人机看不到前进方向的障碍物而炸机。

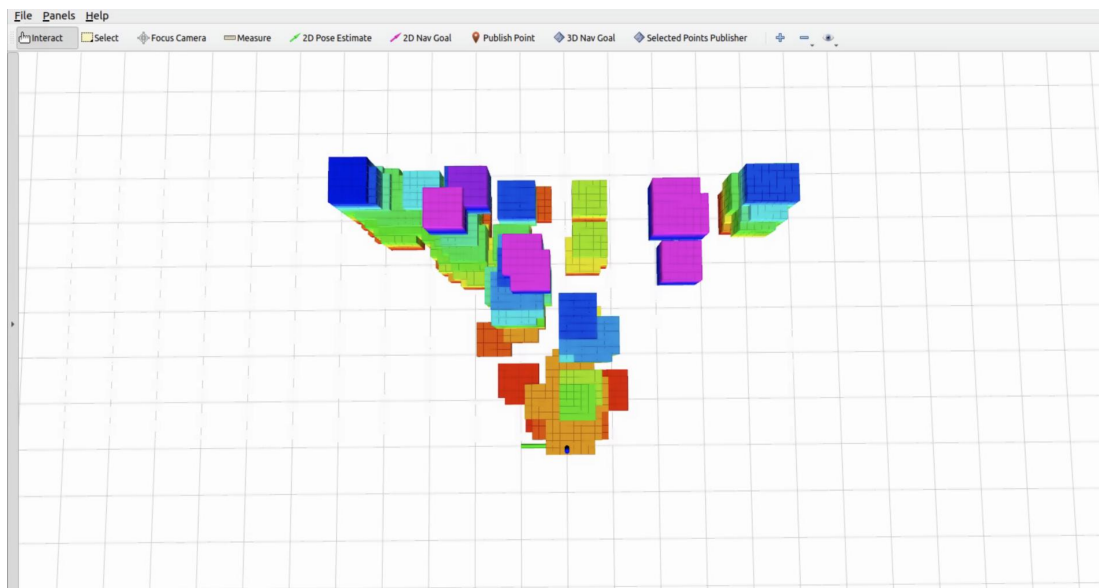
步骤：

1) 参照按 3.1.1 节检查规划和建图参数以及遥控器拨杆是否到位，由于视觉定位容易不稳定，建议最大飞行速度不大于 2m/s。

2) 进入 Diff-Planner 工作空间启动视觉定位规划脚本

```
cd ~/Diff-Planner
./sh_files/run_single_vio.sh
```


程序启动后 rviz 界面如图所示：



3) 定位稳定性检查（可省略，但建议增加这一步）：拿起飞机晃动几下后放回原地，查看定位偏移是否在合理范围内（一般在 0.3m 以内）。

4) 参照 3.1.1 节中的执行任务方式执行任务。

3.2 Elastic-Tracker 自主目标跟踪（选配功能）

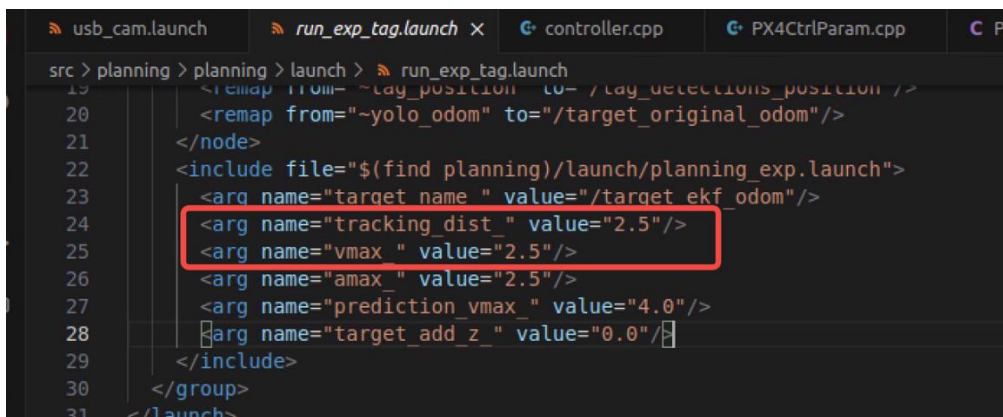
目标：

- 1) 完成 Elastic-Tracker 自主目标跟踪功能测试。
- 2) 熟悉自主目标跟踪参数。

进行前请先完成 2.4 节雷达定位测试和 2.6 节单目视觉检测测试，确保都没有问题再进行！

步骤：

- 1) 检查 src/planning/planning/launch/run_exp_tag.launch 中各追踪参数是否设置合适。



```
src > planning > planning > launch > run_exp_tag.launch
19 <!-- remap from ~/tag_position to ~/tag_detections_position -->
20 <remap from="~/tag_position" to="/tag_detections_position"/>
21 </node>
22 <include file="$(find planning)/launch/planning_exp.launch">
23 <arg name="target_name" value="/target_ekf_odom"/>
24 <arg name="tracking_dist" value="2.5"/>
25 <arg name="vmax" value="2.5"/>
26 <arg name="amax" value="2.5"/>
27 <arg name="prediction_vmax" value="4.0"/>
28 <arg name="target_add_z" value="0.0"/>
29 </include>
30 </group>
31 </launch>
```

run_exp_tag.launch 关键参数表		
参数名	参数值	说明
tracking_dist	2.5	期望追踪距离
vmax	2.5	最大追踪速度
amax	2.5	最大追踪加速度

prediction_vmax_	4.0	目标最大预测速度
target_add_z_	0.0	目标高度偏置

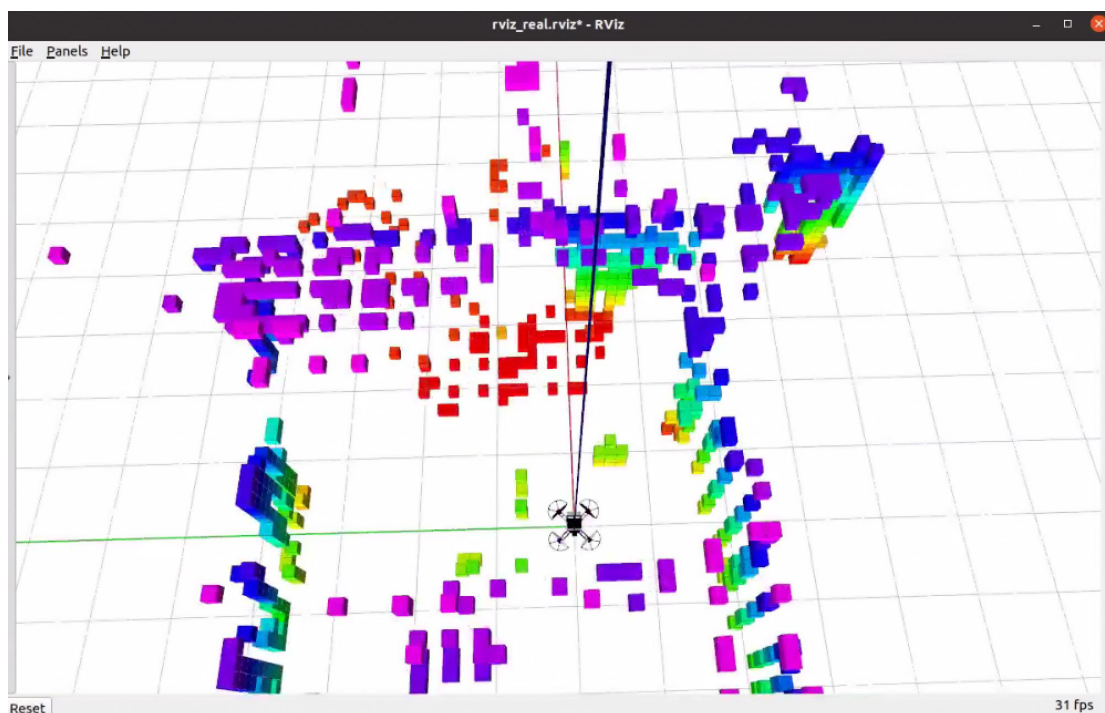
2) 检查 `src/realflight_modules/px4ctrl/config/ctrl_param_fpv.yaml` 中起飞高度 `takeoff_height` 设置是否合适。

3) 将无人机放置于安全、平整地面，以无人机为中心半径 50cm 的四周请勿放置任何障碍物。打开遥控器，保持 5 通、6 通、8 通在最下通道，7 通拨杆在上，油门保持中位。

4) 进入 Elastic-Tracker 工作空间并启动自主跟踪脚本

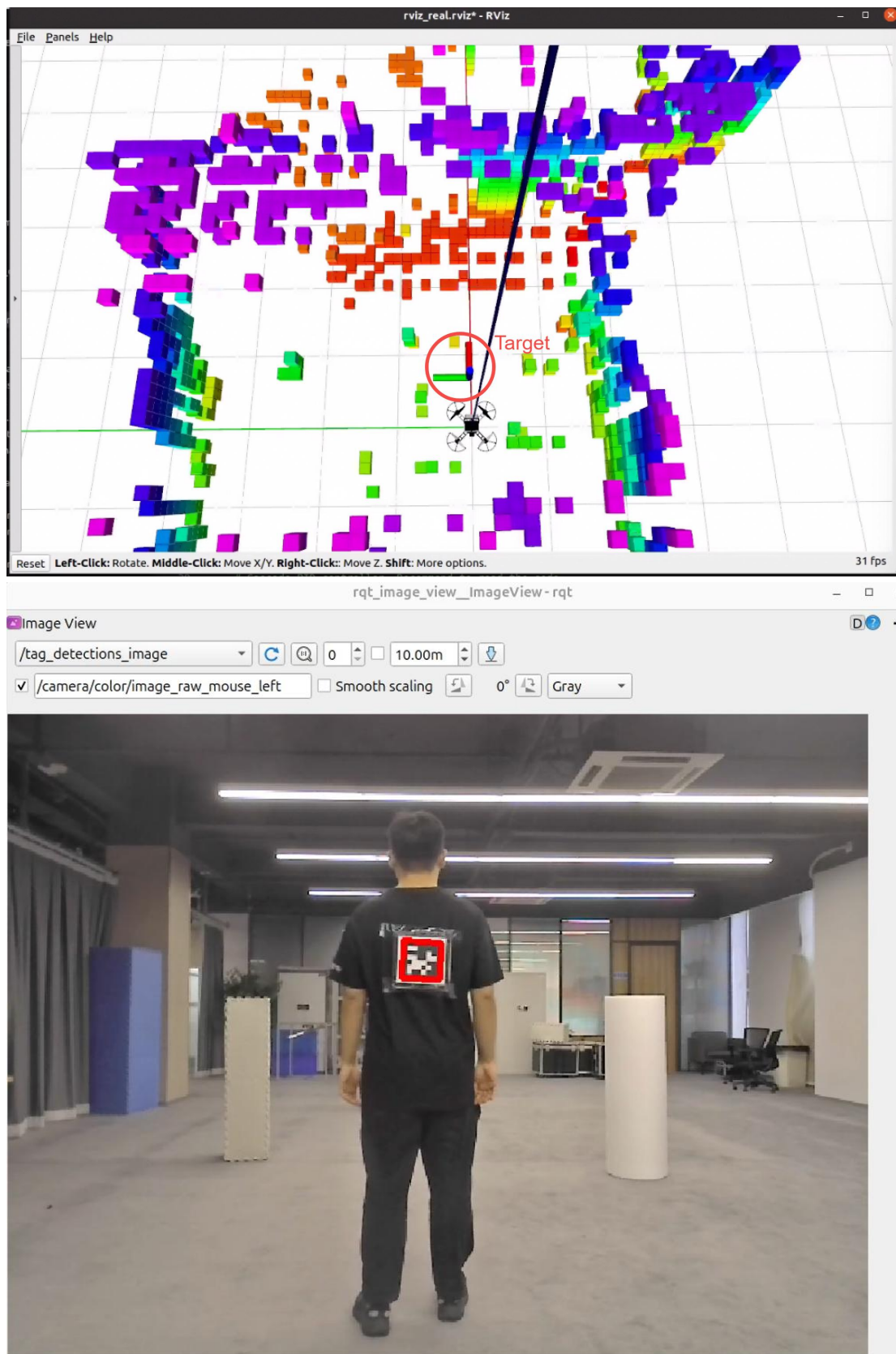
```
cd ~/Diff-Planner
./sh_files/run_all.sh
```

程序启动后 `rviz` 界面所示：



5) 起飞：确定飞机所在环境安全无人员靠近，并再次确定遥控器拨杆状态正常。将 8 通道打到中位，无人机起飞并悬停。此状态下仍可用遥控器操控飞机（雷达定位模式）。

6) 将贴有二维码的目标就位至无人机前适当距离处（期望追踪距离附近），查看 `rviz` 中目标位置是否出现（可通过 `rostopic echo /target_ekf_odom` 查看是否有消息输出），也可打开 `rqt_image_view` 实时查看检测效果。



7) 开始跟踪：确认目标可被识别后，8通道打到上位开始跟踪。**注意：二维码移动过程中尽量使其正对飞机相机，尽量匀速运动，避免急停急走；转弯时注意要慢，确保飞机始终能正视二维码；若跟丢，把二维码重新放回飞机视野中可重新开始追踪。**

8) 跟踪结束后，6通道从下档打至上档，无人机进入悬停。待悬停稳定后，8通道

从上档打至中档，飞机自动降落。降落后 7 通下拨急停上锁，并在任务终端输入 **ctrl+c** 结束程序。**注意：若跟踪过程中出现撞到障碍物、与目标过近等意外，在飞行未失控情况下及时将 6 通打至上档进入悬停。**

3.3 FUEL 自主探索（选配功能）

目标：

- 1) 完成 FUEL 自主探索功能测试。
- 2) 熟悉自主探索参数。

进行前请先完成 2.4 节雷达定位测试，确保没有问题再进行！

步骤：

1) 在 `src/fuel_planner/exploration_manager/launch/exp_exploration.launch` 中设置探索范围。探索坐标系与无人机起飞位置机体系重合，即以起飞位置为原点（可通过设置 `init_x/y/z` 参数来设置偏置），正前方为 `x` 轴正方向。`box_min_x/y/z` 分别为探索范围 `x/y/z` 轴最小值，`box_max_x/y/z` 分别为探索范围 `x/y/z` 轴最大值。

```
exp_exploration.launch x
src > fuel_planner > exploration_manager > launch > exp_exploration.launch
1 <launch>
2 <!-- size of map, change the size in x, y, z according to your application -->
3 <arg name="map_size_x" value="50.0"/>
4 <arg name="map_size_y" value="50.0"/>
5 <arg name="map_size_z" value="10.0"/>
6
7 <arg name="init_x" value="0"/>
8 <arg name="init_y" value="0"/>
9 <arg name="init_z" value="1.2"/>
10
11 <!-- topic of your odometry such as VIO or LIO -->
12 <arg name="odom_topic" value="/ekf/ekf_odom" />
13
14 <!-- main algorithm params -->
15 <include file="$(find exploration_manager)/launch/exp_algorithm.xml">
16
17   <arg name="map_size_x" value="$(arg map_size_x)"/>
18   <arg name="map_size_y" value="$(arg map_size_y)"/>
19   <arg name="map_size_z" value="$(arg map_size_z)"/>
20
21   <arg name="box_min_x" value="-0.5"/>
22   <arg name="box_max_x" value="15.0"/>
23   <arg name="box_min_y" value="-4.0"/>
24   <arg name="box_max_y" value="4.0"/>
25   <arg name="box_min_z" value="-0.1"/>
26   <arg name="box_max_z" value="1.5"/>
27
28   <arg name="odometry_topic" value="$(arg odom_topic)"/>
29   <!-- sensor pose: transform of camera frame in the world frame -->
30   <arg name="sensor_pose_topic" value="/laserMapping/odometry"/>
31   <!-- cloud topic: point cloud measurement -->
32   <!-- subscribe ONLY TO ONE of the two topics -->
33   <arg name="cloud_topic" value="/laserMapping/cloud_registered"/>
34   <!-- maximum velocity and acceleration the drone will reach -->
35   <arg name="max_vel" value="1.0" />
36   <arg name="max_acc" value="1.0" />
37 </include>
```

exp_exploration.launch 关键参数表

参数名	参数值	说明
map_size_x	50	地图尺寸 x 轴方向（不小于探索范围 x 轴方向）

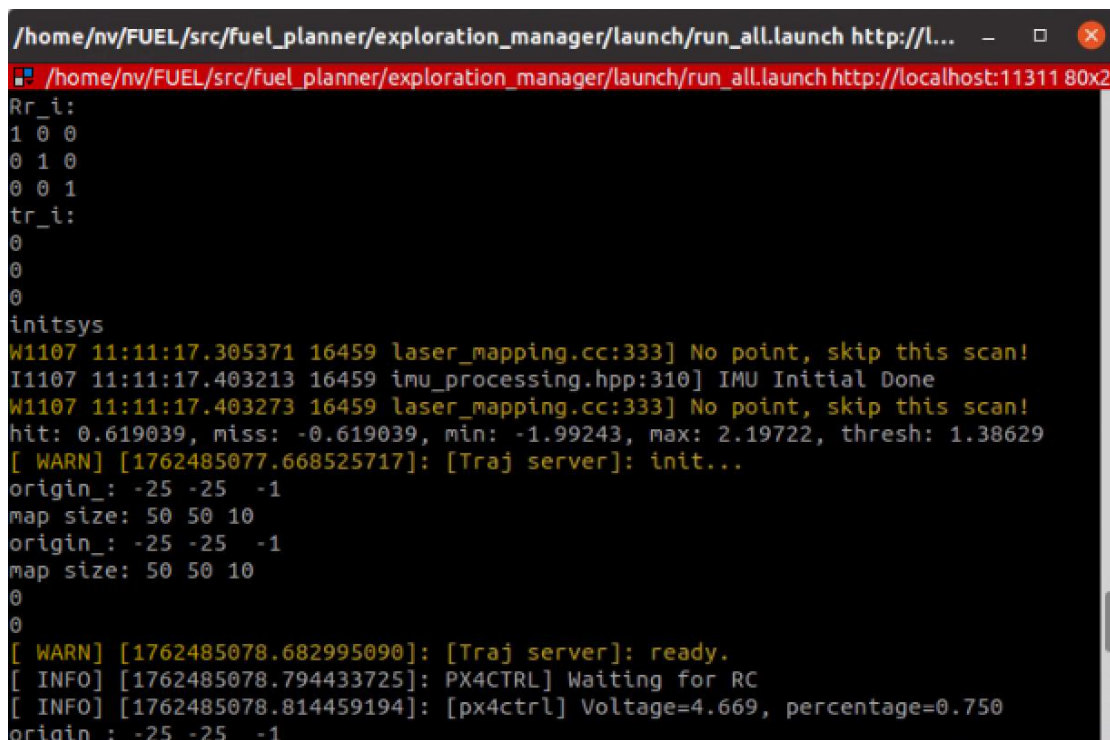
map_size_x	50	地图尺寸 y 轴方向（不小于探索范围 y 轴方向）
map_size_z	10	地图尺寸 z 轴方向（不小于探索范围 z 轴方向）
init_x	0.0	探索范围原点相对于起飞点 x 轴偏置
init_y	0.0	探索范围原点相对于起飞点 y 轴偏置
init_z	1.2	探索范围原点相对于起飞点 z 轴偏置
box_min_x	-0.5	探索范围 x 轴最小值
box_max_x	15.0	探索范围 x 轴最大值
box_min_y	-4.0	探索范围 y 轴最小值
box_max_y	4.0	探索范围 y 轴最大值
box_min_z	-0.1	探索范围 z 轴最小值
box_max_z	1.5	探索范围 z 轴最大值
max_vel	1.0	最大探索速度
max_acc	1.0	最大探索加速度

2) 将无人机放置于安全、平整地面，以无人机为中心半径 50cm 的四周请勿放置任何障碍物。打开遥控器，保持 5 通、6 通、8 通在最下通道，7 通拨杆在上，油门保持中位。

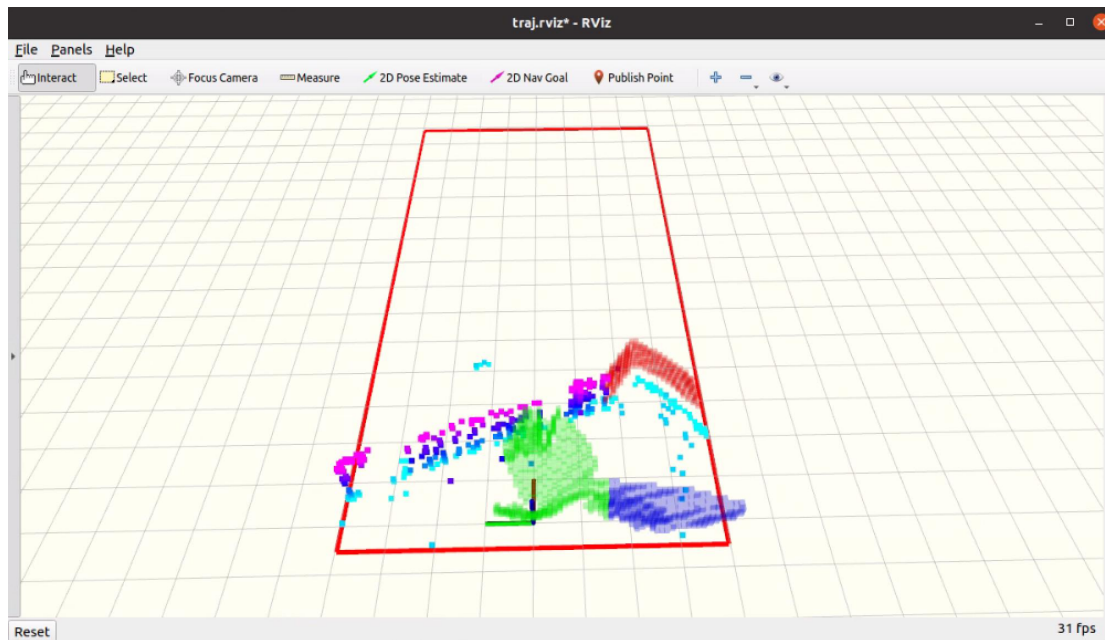
3) 进入 FUEL 工作空间并启动自主探索脚本

```
cd ~/FUEL
./sh_files/run_all.sh
```

程序启动后终端和 rviz 可视化软件如图所示：



```
/home/nv/FUEL/src/fuel_planner/exploration_manager/launch/run_all.launch http://l...
/home/nv/FUEL/src/fuel_planner/exploration_manager/launch/run_all.launch http://localhost:11311 80x24
Rr_i:
1 0 0
0 1 0
0 0 1
tr_i:
0
0
0
initsys
W1107 11:11:17.305371 16459 laser_mapping.cc:333] No point, skip this scan!
I1107 11:11:17.403213 16459 imu_processing.hpp:310] IMU Initial Done
W1107 11:11:17.403273 16459 laser_mapping.cc:333] No point, skip this scan!
hit: 0.619039, miss: -0.619039, min: -1.99243, max: 2.19722, thresh: 1.38629
[ WARN] [1762485077.668525717]: [Traj server]: init...
origin_: -25 -25 -1
map size: 50 50 10
origin_: -25 -25 -1
map size: 50 50 10
0
0
[ WARN] [1762485078.682995090]: [Traj server]: ready.
[ INFO] [1762485078.794433725]: PX4CTRL] Waiting for RC
[ INFO] [1762485078.814459194]: [px4ctrl] Voltage=4.669, percentage=0.750
origin : -25 -25 -1
```



4) 确定飞机所在环境安全无人员靠近，并再次确定遥控器拨杆状态正常将 8 通道打到中位，无人机起飞并悬停。待无人机悬停稳定后，将 8 通道上拨开始探索任务。

5) 结束探索任务后，把 6 通道从下档拨至上档，无人机进入自动悬停模式。待无人机悬停稳定后，把 8 通道从上档拨至下档，无人机自动降落。落地之后将 7 通下拨，无人机锁桨，并在任务终端输入 `ctrl+c` 结束程序。

3.4 Formation 集群（选配功能）

目标：

- 1) 完成 formation 集群功能测试（以四机集群为例）。
- 2) 熟悉 formation 集群参数。

进行前请先完成 2.4 节雷达定位测试，确保都没有问题再进行！

步骤：

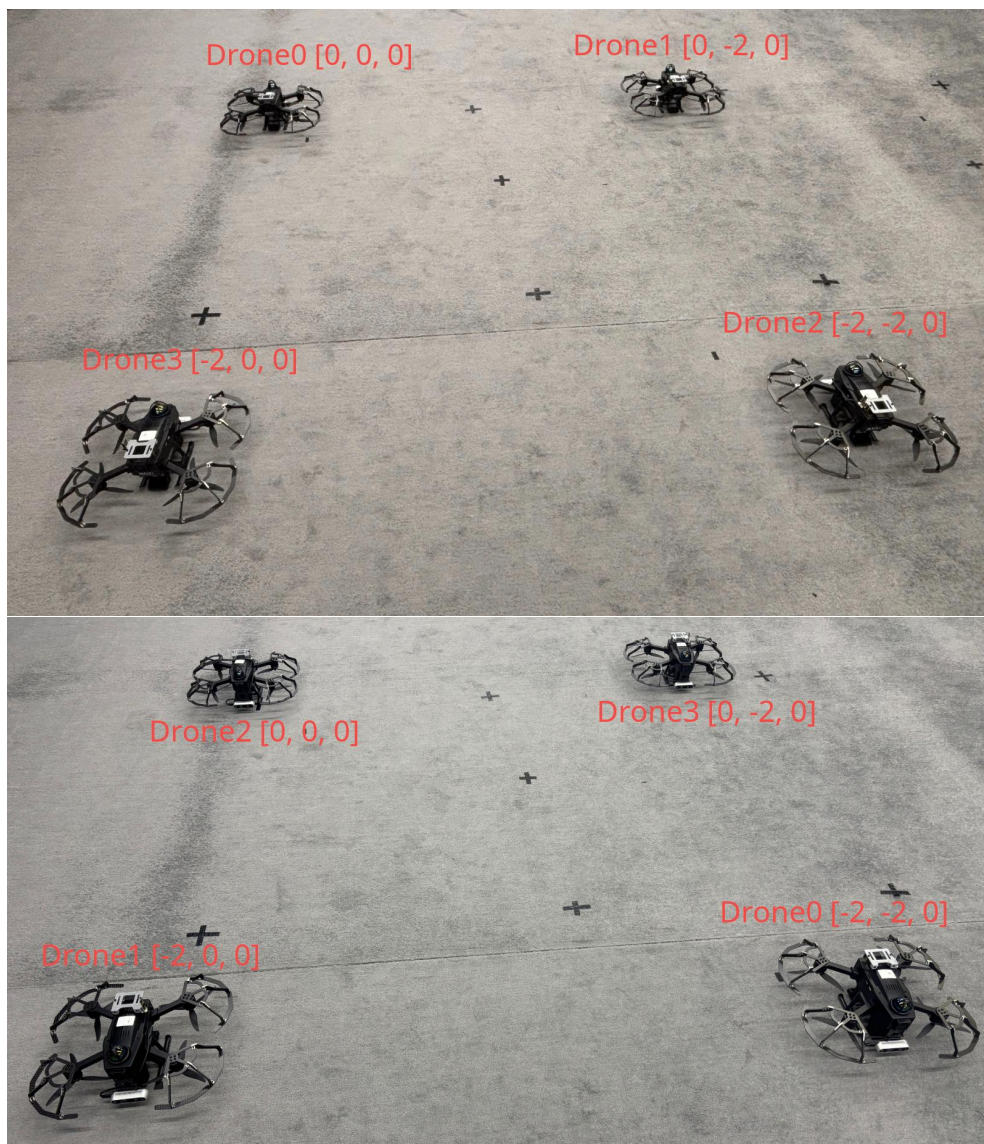
- 1) 将所有飞机和笔记本地面站连接到同一局域网下，建议固定所有设备 ip
- 2) 设置所有无人机的集群参数。在 `src/planner/plan_manage/launch/run_swarm.launch` 中设置航点坐标；在 `src/planner/plan_manage/launch/advanced_param_real.xml` 中设置集群相对位置。需要设置的各个参数含义如下表所示：

run_swarm.launch 关键参数表		
参数名	参数值	说明
flight_type_	2	飞行模式，2 代表起飞后自动执行任务
point0_x	8.5	第一个航点 x 坐标
point0_y	0.0	第一个航点 y 坐标
point0_z	1.0	第一个航点 z 坐标
point1_x	-2.0	第二个航点 x 坐标
point1_y	-2.0	第二个航点 y 坐标
point1_z	1.0	第二个航点 z 坐标
advanced_param_real.xml 关键参数表		
参数名	参数值	说明

formation/num	4	集群数量
formation/drone0	[0, 0, 0]	0 号机初始位置
formation/drone1	[0.0, -2.0, 0.0]	1 号机初始位置
formation/drone2	[-2.0, -2.0, 0.0]	2 号机初始位置
formation/drone3	[-2.0, 0.0, 0]	3 号机初始位置

注意航点按照 0 号机的期望航点来设置，所有飞机的航点都设置为和 0 号机相同，他们在执行时的实际航点会根据初始位置参数 formation/drone* 自动做偏置，使得在每个航点处所有飞机的相对位置保持和初始相对位置相同。

设置示例：在本例中，0-3 号无人机的起飞位置分别为 $\{0,0,0\}$ $\{0,-2,0\}$ $\{-2,-2,0\}$ $\{-2,0,0\}$ ，如下左图所示。0 号机第一个航点为 $\{8.5,0.0,1.0\}$ ，飞往该点之后集群设置的返回航点的不是原点 $\{0,0,0\}$ ，而是 $\{-2,-2,1\}$ ，这是因为在变换队形的时候围绕 0 号保持编队，如果设置返航的点为原点，最终 0-3 号飞机最终将会降落在 $\{0,0,0\}$ $\{0,2,0\}$ $\{2,2,0\}$ $\{2,0,0\}$ ，而将返回的航点设置为 $\{-2,-2,1\}$ 能保证四架飞机整体上处于出发的位置，如下右图所示。



注意：在启动程序前，请将所有无人机按照设置的初始位置摆放（使用卷尺尽可能精确测量相对位置），并尽可能将无人机 yaw 角对准 x 轴方向。

3) 设置所有无人机的初始位置偏置。由于无人机里程计在初始化时位置默认为[0,0,0], 需要在 src/realflight_modules/ekf_pose/launch/ekf_lidar.launch 中设置 offset_px/offset_py/offset_pz, 将其设置为和自己的初始位置相同, 即和自己编号对应的 formation/drone*参数相同。

```
ekf_lidar.launch x
src > realflight_modules > ekf_pose > launch > ekf_lidar.launch
10
11 <!-- params -->
12 <rosparam file="$(find ekf)/launch/PX4_vio_drone.yaml" command="load" />
13
14 <!-- initial pos offset before pub -->
15 <param name="offset_px" type="double" value="0.0"/>
16 <param name="offset_py" type="double" value="0.0"/>
17 <param name="offset_pz" type="double" value="0.0"/>
18
19 <!-- body in IMU frame -->
20 <param name="imu_trans_x" type="double" value="0.0"/>
21 <param name="imu_trans_y" type="double" value="0.0"/>
22 <!-- <param name="imu_trans_z" type="double" value="0.0"/> -->
23 <param name="imu_trans_z" type="double" value="0.0"/>
24
```

4) 设置所有无人机的通讯参数。检查 src/planner/swarm_bridge/launch/bridge_tcp_drone.launch 中参数配置是否正确:

bridge_tcp_drone.launch 关键参数说明		
参数名	参数值	说明
drone_id	0	无人机编号
drone_num	4	集群数量
drone_ip_0	192.168.60.120	0号飞机的IP
drone_ip_1	192.168.60.121	1号飞机的IP
drone_ip_2	192.168.60.122	2号飞机的IP
drone_ip_3	192.168.60.123	3号飞机的IP
ground_station_num	1	地面站的数量
ground_station_ip_0	192.168.60.126	地面站的IP

5) 在地面站使用 terminator 打开终端, 将其拆分为 3*4 个终端, 每一列使用 ssh 连接同一架无人机终端, 从左到右分别连接 0-3 号无人机终端, 并将每一行终端分为一组方便后续操作:



6) 时间同步。首先, 在无人机 home 目录下的 ntpdate.sh 脚本文件中设置要同步的

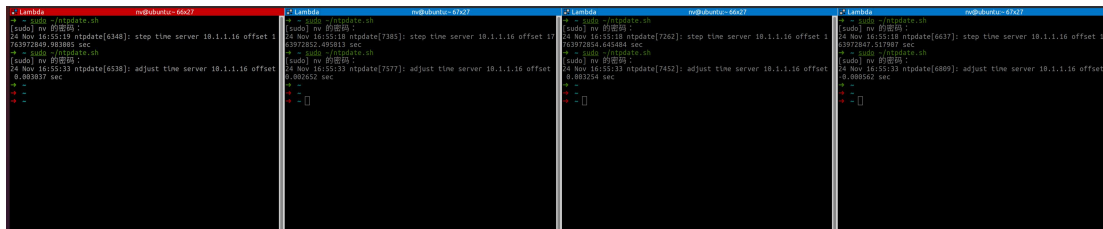
主机的 IP，将其改为自己地面站电脑 IP，需要保证所有的集群飞机同步的是同一台：

```
ntpdate -u 192.168.60.126
```

使用 terminator 的广播组功能，在第一行输入：

```
sudo ~/ntpdate.sh
```

时间同步的结果：



注意：时间同步的单位为秒，尽量让所有飞机的时间同步的 offset 为毫秒级别（即 0.00x），有利于集群的稳定性。如果误差较大就多运行几次该脚本。

7) 打开遥控器，检查通道是否正确：5 通、6 通、8 通在最下，7 通在上，油门保持中位。

8) 启动地面站监测。在地面站笔记本端启动地面站程序用于检测无人机状态。首先将无人机上 formation_ws 中 src 文件夹拷贝至地面站自己的工作空间，并删除 realflight_modules 组件，之后使用 catkin_make 编译。接下来在 src/planner/swarm_bridge/launch/bridge_tcp_station.launch 中配置地面站和无人机 IP：

run_swarm.launch 关键参数表		
参数名	参数值	说明
drone_num	4	集群数量
drone_ip_0	192.168.60.120	0 号飞机的 IP
drone_ip_1	192.168.60.121	1 号飞机的 IP
drone_ip_2	192.168.60.122	2 号飞机的 IP
drone_ip_3	192.168.60.123	3 号飞机的 IP
ground_station_num	1	地面站的数量
ground_station_ip_0	192.168.60.126	地面站的 IP

配置完该文档之后即可开始使用地面站：

```
source devel/setup.zsh
roslaunch swarm_bridge bridge_tcp_station.launch
source devel/setup.zsh #新建终端
roslaunch ego_planner rviz.launch
```

9) 启动无人机集群程序。使用 terminator 的广播组功能，在第一行终端预输入启动脚本（输完启动脚本先不按回车执行）：

```
cd formation_ws
./sh_files/run_all.sh
```

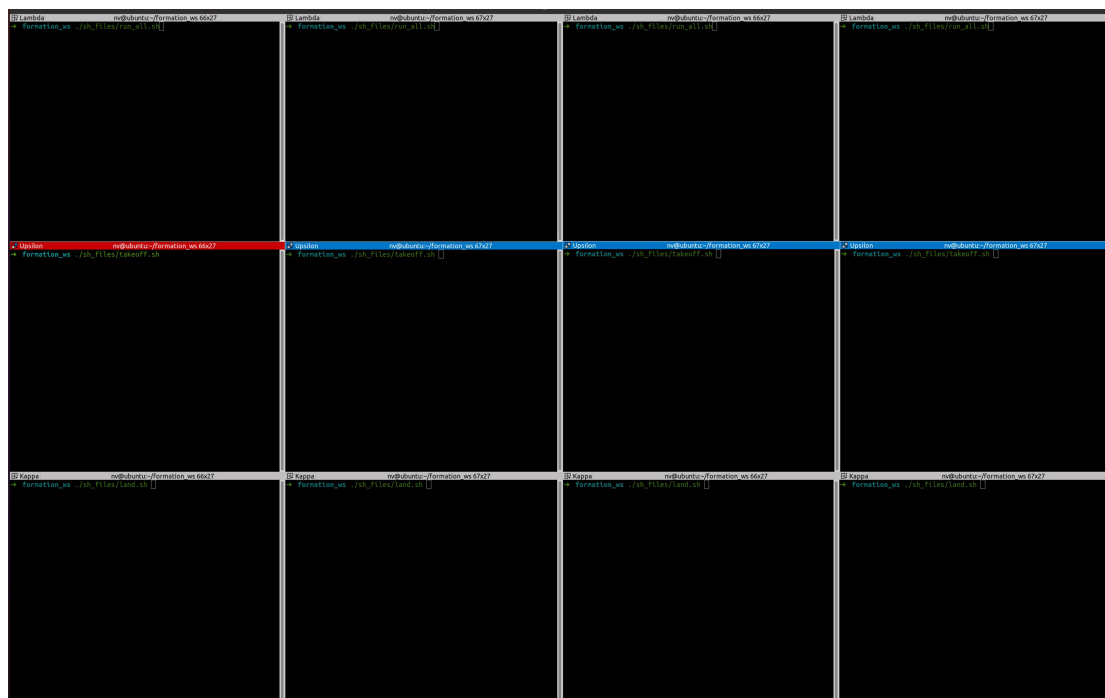
在第二行终端预输入起飞脚本（输完起飞脚本先不按回车执行）：

```
cd formation_ws
./sh_files/takeoff.sh
```

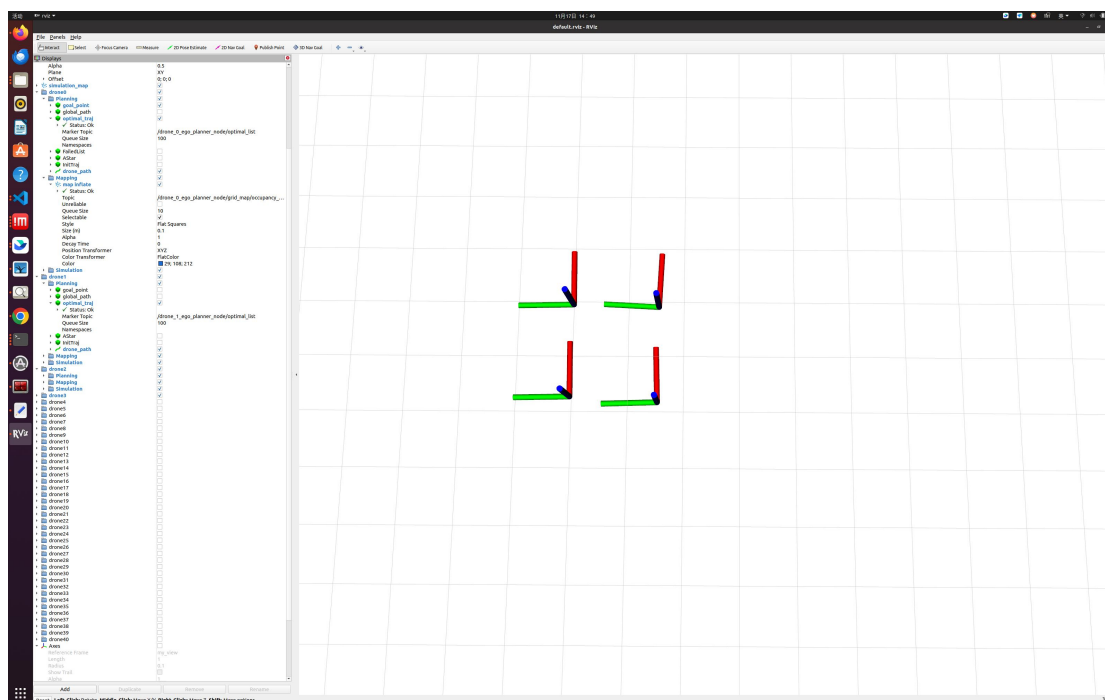
在第三行终端预输入降落脚本（输完降落脚本先不按回车执行）：

```
cd formation_ws
./sh_files/land.sh
```

终端命令如下所示：



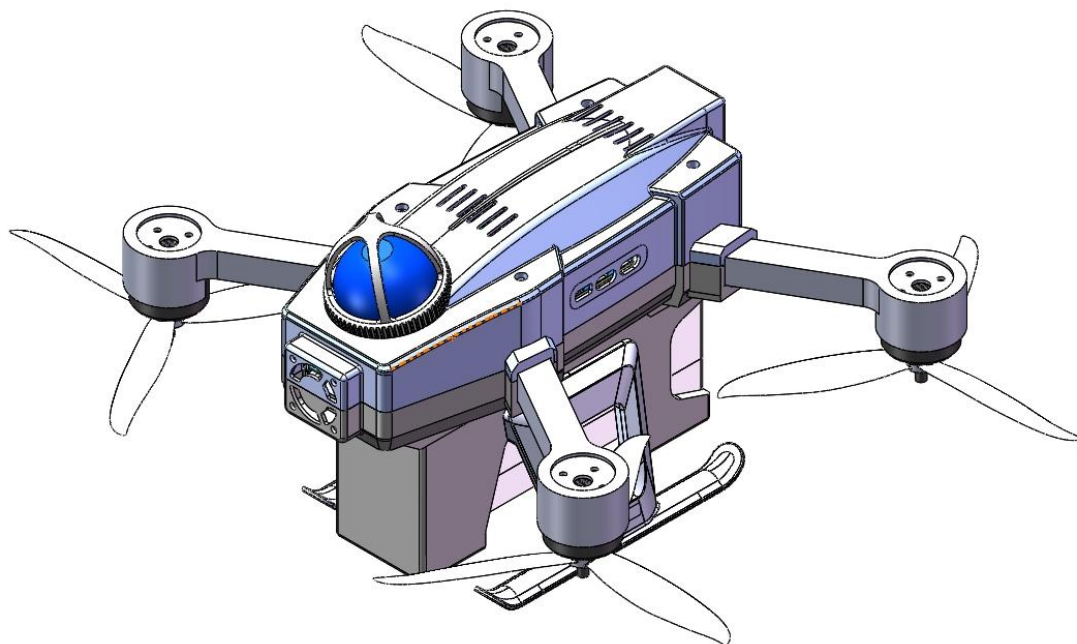
之后在第一个终端按回车启动集群程序，等待约 30s 以后程序启动完成。此时在地面站的 Rviz 中可以看见四架无人机的位置（四架无人机坐标会依次快速闪烁，因为使用同一个话题同时发布四架无人机位置）：



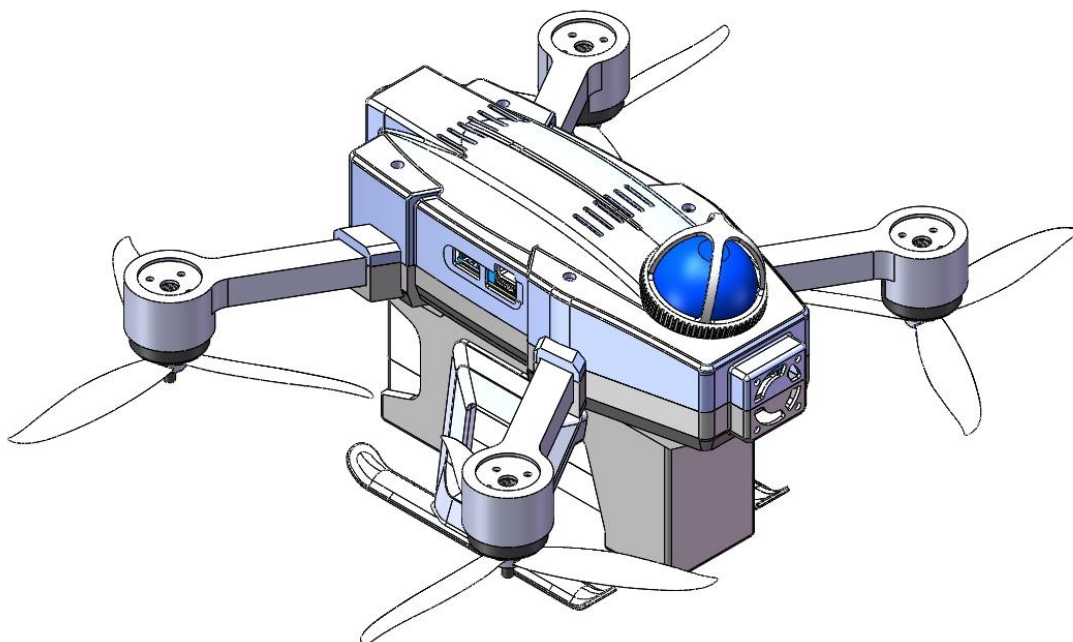
10) 起飞并开始集群规划。起飞前请再次确认所有遥控器拨杆是否正确，集群程序和地面站程序也正常启动。在地面站第二行终端按回车启动起飞脚本，之后四台飞机将会同时起飞，在起飞到预设高度后将会自动开始集群规划。

11) 降落。确认所有飞机到达最后一个目标点进入悬停之后，在地面站第三行终端按回车启动降落脚本，四架飞机将会自动降落，待落地后将所有遥控器 7 通下拨急停上锁。

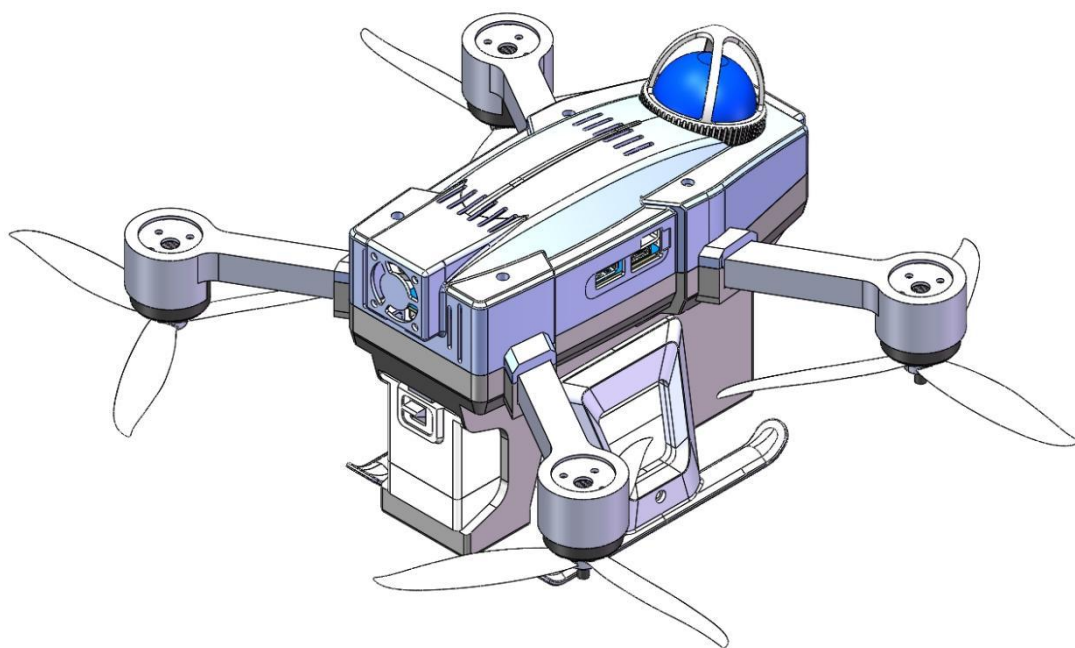
附件一 非凸 α 硬件结构图



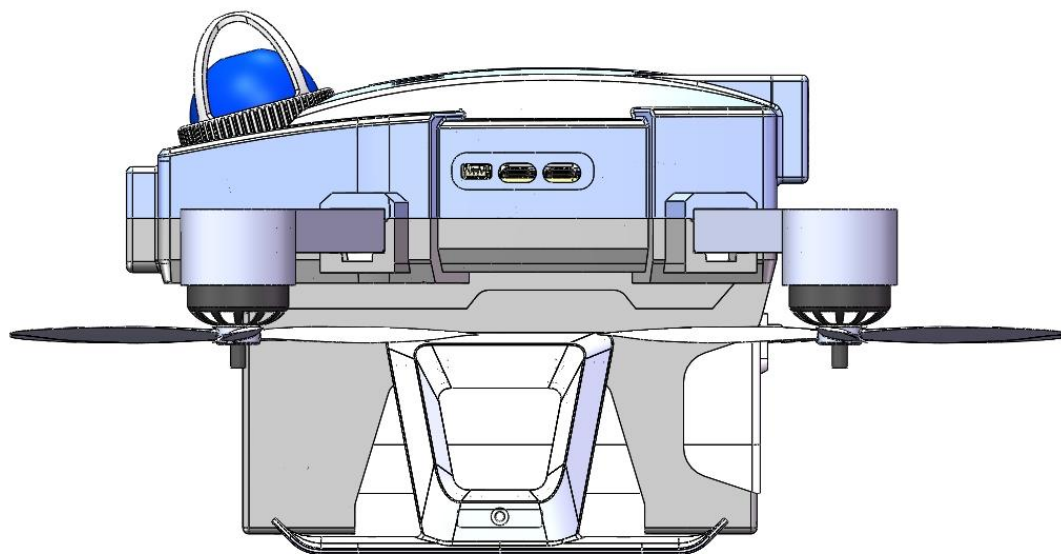
(图一 前视右上方轴测图)



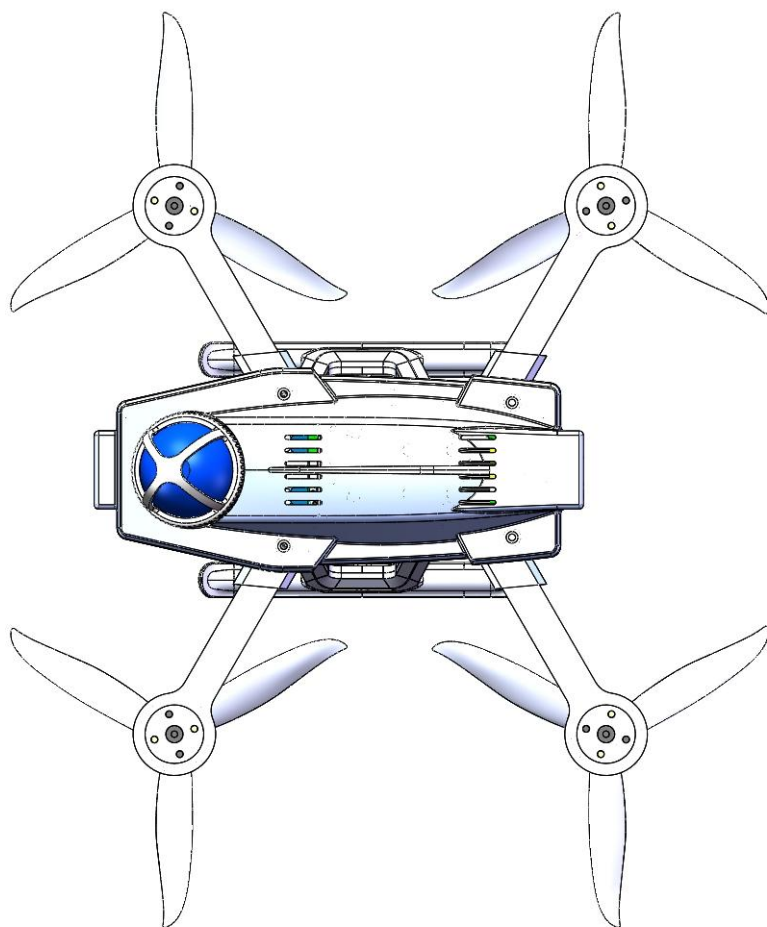
(图二 前视左上方轴测图)



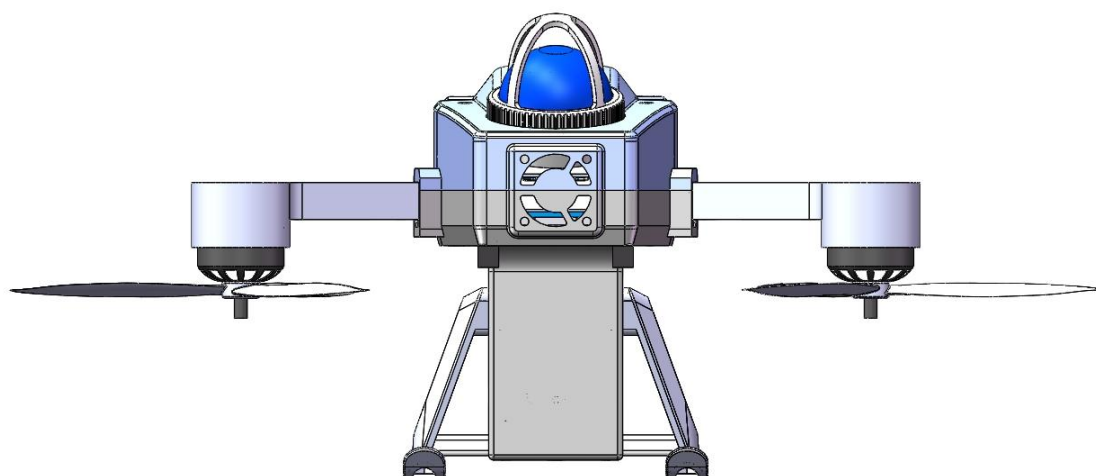
(图三 后视右上方轴测图)



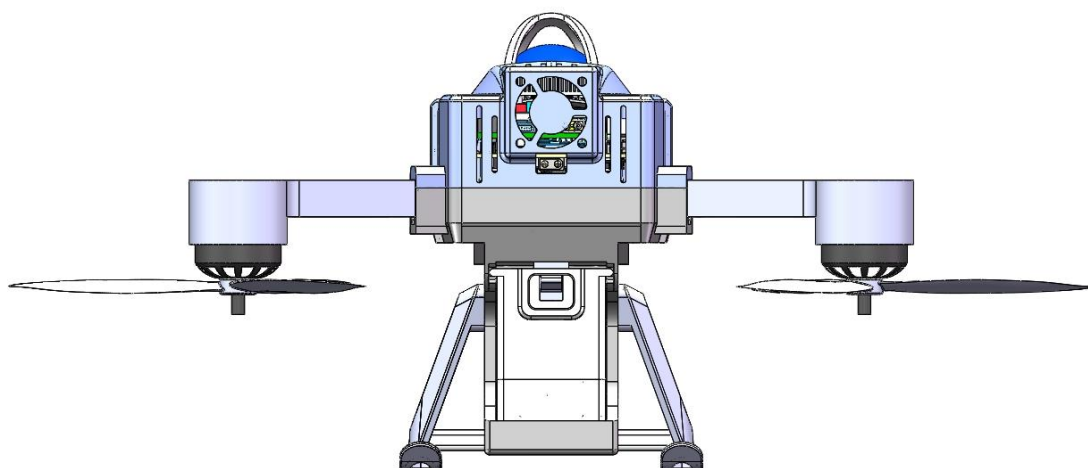
(图四 无人机左视图)



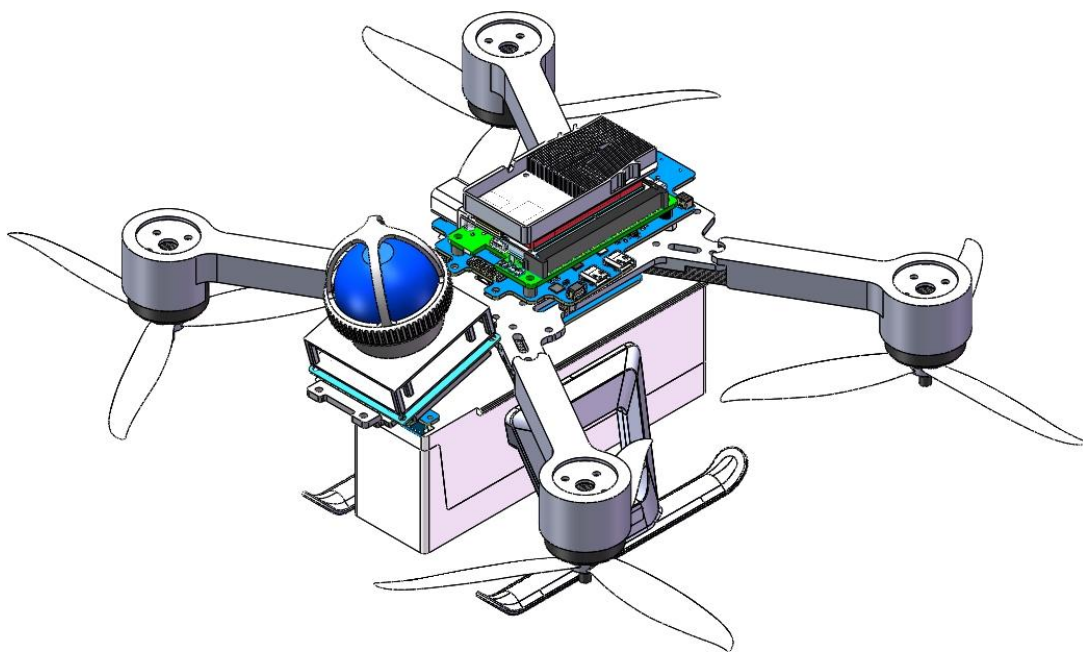
(图五 无人机上视图)



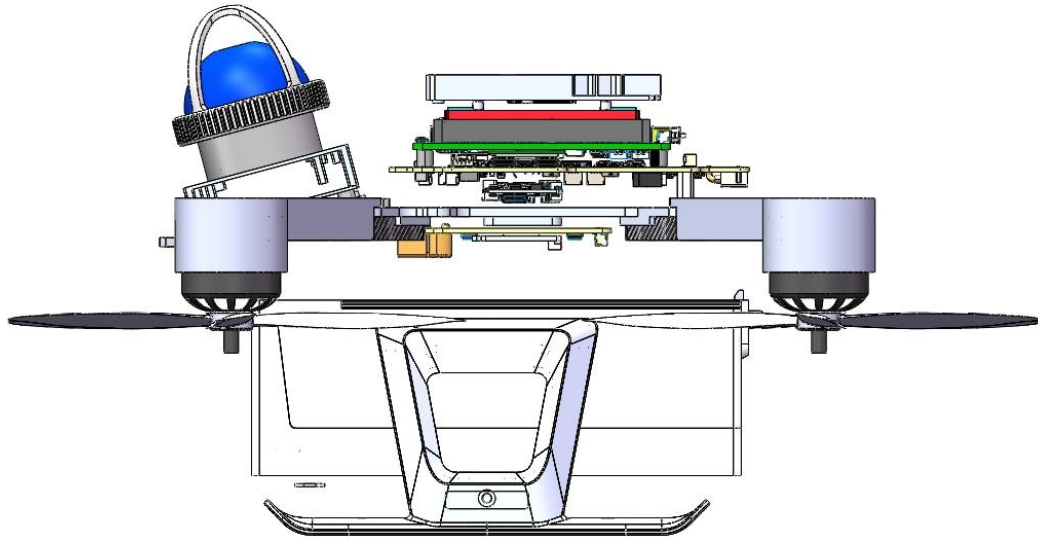
(图六 无人机前视图)



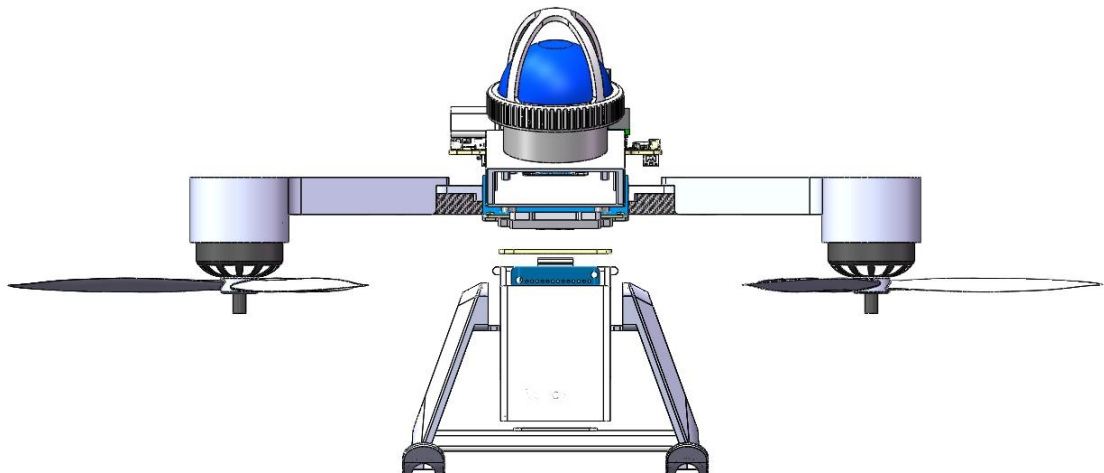
(图七 无人机后视图)



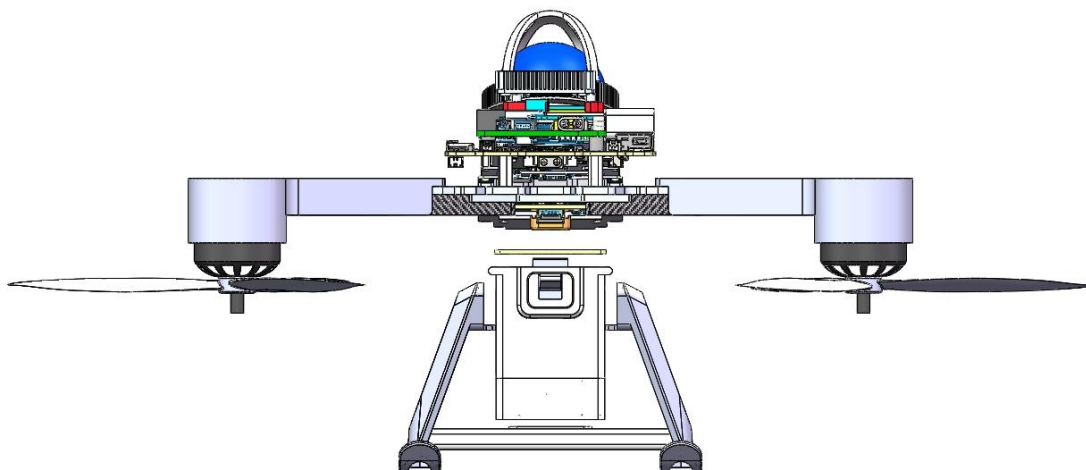
(图八 无人机左上轴侧剖面图)



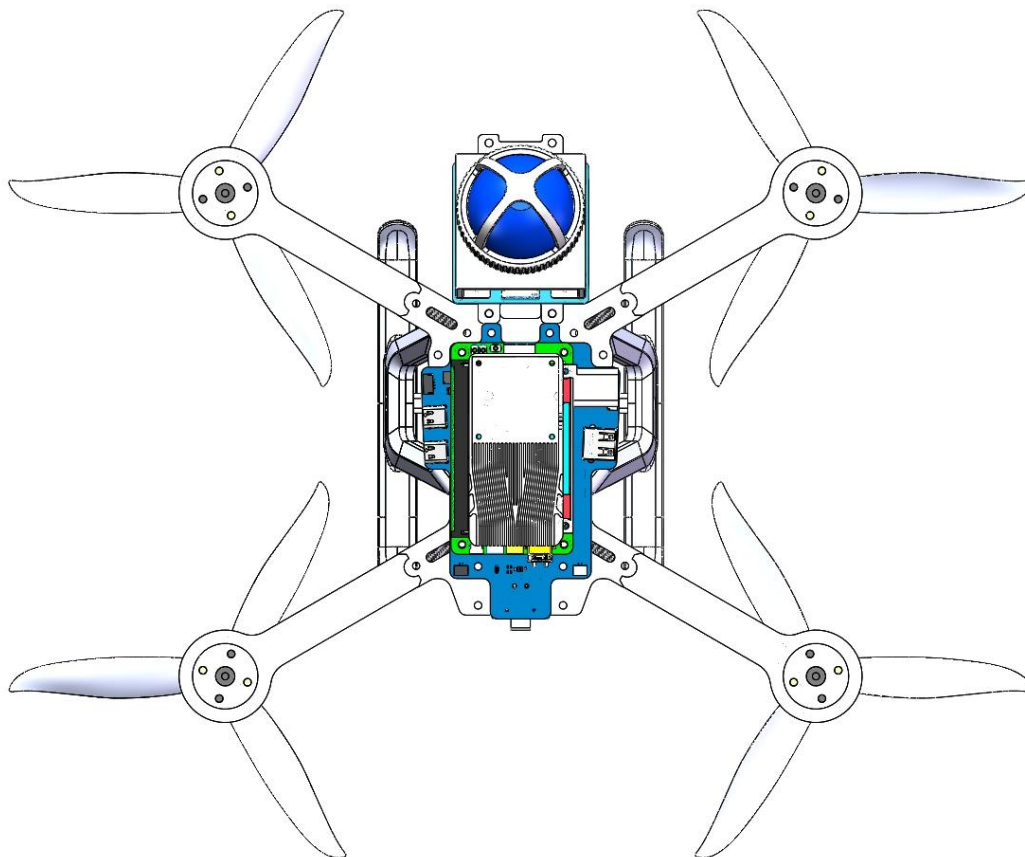
(图九 无人机左视剖面图)



(图十 无人机前视剖面图)



(图十一 无人机后视剖面图)



(图十二 无人机上视剖面图)

附件二 相关算法资料链接

1. Faster-LIO: <https://github.com/gaoxiang12/faster-lio>
2. VINS-Fusion-gpu: <https://github.com/pjrambo/VINS-Fusion-gpu>
3. Diff-Planner: <https://github.com/DifferentialRobotics/Diff-Planner>
4. Elastic-Tracker: <https://github.com/ZJU-FAST-Lab/Elastic-Tracker>
5. FUEL: <https://github.com/HKUST-Aerial-Robotics/FUEL>
6. Formation: <https://github.com/ZJU-FAST-Lab/EGO-Planner-v2>

附件三 tag36h11

